

Helicóptero



Contents

1	Data structures	1
2	Graphs	5
3	Math	9
4	Geometry	17
5	Strings	21
6	Flow	23
7	Other	26

1 Data structures

1.1 Segment tree

```
1 typedef ll node;
2 #define oper min
3 #define NEUT INF
4 struct STree{
5     int n; vector<node>t;
6     STree(int n):n(n),t(2*n+5,NEUT){}
7     void init(vector<node> &a){
8         fore(i,0,n)t[n+i]=a[i];
9         for(int i=n-1;i>0;i--)t[i]=oper(t[2*i],t[2*i
10             +1]);
11     }
12     void upd(int p, node v){
13         for(p+=n,t[p]=v;p>1;p>>=1)p|=1,t[p>>1]=oper(t[p
14             ^1],t[p]);
15     }
16     node query(int l, int r){
17         node izq=NEUT,der=NEUT;
18         for(l+=n,r+=n;l<r;l>>=1,r>>=1){
19             if(l&1)izq=oper(izq,t[l++]);
20             if(r&1)der=oper(t[--r],der);
21         }
22         return oper(izq,der);
23     }
24 };
25 
```

1.2 Segment tree - Lazy

```
1 // example: range sum with range addition
2 typedef ll tn; // segment-tree node type
3 typedef ll tl; // lazy type
4 #define NEUT 0 // operation neutral
5 #define CLEAR 0 // cleared lazy node
6 tn oper(tn a, tn b){return a+b;}
7 void acum(tl &a, tl v){a+=v;}
8 tn calc(int s, int e, tn a, tl v){a+=(e-s)*v;
9     return a;}
10 struct STree{
11 
```

```
12 vector<tn>st; vector<tl>lazy; int n;
13 STree(int n):st(4*n+5,NEUT),lazy(4*n+5,CLEAR),n(n)
14     {}
15 void init(int k, int s, int e, vector<tn>&a){
16     if(e-s==1)st[k]=a[s];
17     else {
18         int m=(s+e)/2;
19         init(2*k,s,m,a); init(2*k+1,m,e,a);
20         st[k]=oper(st[2*k],st[2*k+1]);
21     }
22 }
23 void push(int k, int s, int e){
24     if(lazy[k]==CLEAR)return;
25     st[k]=calc(s,e,st[k],lazy[k]);
26     if(e-s!=1){
27         acum(lazy[2*k],lazy[k]);
28         acum(lazy[2*k+1],lazy[k]);
29     }
30     lazy[k]=CLEAR;
31 }
32 void upd(int k, int s, int e, int a, int b, tl v)
33     {
34     push(k,s,e);
35     if(e<=a||b<=s)return;
36     if(a<=s&&b<=e){
37         acum(lazy[k],v);
38         push(k,s,e);
39         return;
40     }
41     int m=(s+e)/2;
42     upd(2*k,s,m,a,b,v),upd(2*k+1,m,e,a,b,v);
43     st[k]=oper(st[2*k],st[2*k+1]);
44 }
45 tn query(int k, int s, int e, int a, int b){
46     if(e<=a||b<=s)return NEUT;
47     push(k,s,e);
48     if(a<=s&&b<=e)return st[k];
49     int m=(s+e)/2;
50     return oper(query(2*k,s,m,a,b),query(2*k+1,m,e,
51         a,b));
52 }
53 void upd(int a, int b, tl v){upd(1,0,n,a,b,v);}
54 tn query(int a, int b){return query(1,0,n,a,b);}
55 void init(vector<tn>&a){init(1,0,n,a);}
56 };
57 
```

1.3 Segment tree - Persistence

```
1 struct STree { //persistent segment tree for min
2     vector<int> st, L, R; int n,sz,rt;
3     STree(int n): st(1,NEUT),L(1,0),R(1,0),n(n),rt(0)
4         ,sz(1){}
5     int new_node(int v, int l=0, int r=0){
6         int ks=SZ(st); st.pb(v); L.pb(l); R.pb(r);
7         return ks;
8     }
9     void upd(int k, int s, int e, int p, int v){
10         int ks=new_node(st[k],L[k],R[k]);
11         if(s+1==e){st[ks]=v;return ks;}
12         int m=(s+e)/2,ps;
13         if(p<m)ps=upd(L[ks],s,m,p,v),L[ks]=ps;
14         else ps=upd(R[ks],m,e,p,v),R[ks]=ps;
15         st[ks]=oper(st[L[ks]],st[R[ks]]);
16         return ks;
17     }
18 
```

```

15 }
16 int query(int k, int s, int e, int a, int b){
17     if(e<=a||b<=s)return NEUT;
18     if(a<=s&&e<=b)return st[k];
19     int m=(s+e)/2;
20     return oper(query(L[k],s,m,a,b),query(R[k],m,e,
        a,b));
21 }
22 int upd(int k, int p, int v){return rt=upd(k,0,n,
        p,v);}
23 int upd(int p, int v){return upd(rt,p,v);} //
        update on last root
24 int query(int k,int a, int b){return query(k,0,n,
        a,b);}
25 };
    
```

1.4 Segment tree - 2D

```

1 int n,m; int a[MAXN][MAXN],st[2*MAXN][2*MAXN];
2 void build(){
3     fore(i,0,n)fore(j,0,m)st[i+n][j+m]=a[i][j];
4     fore(i,0,n)for(int j=m-1;j-->0)
5         st[i+n][j]=op(st[i+n][j<<1],st[i+n][j<<1|1]);
6     for(int i=n-1;i-->0)fore(j,0,2*m)
7         st[i][j]=op(st[i<<1][j],st[i<<1|1][j]);
8 }
9 void upd(int x, int y, int v){
10     st[x+n][y+m]=v;
11     for(int j=y+m;j>1;j>>=1)st[x+n][j>>1]=op(st[x+n][
        j],st[x+n][j^1]);
12     for(int i=x+n;i>1;i>>=1)for(int j=y+m;j>>=1)
13         st[i>>1][j]=op(st[i][j],st[i^1][j]);
14 }
15 int query(int x0, int x1, int y0, int y1){
16     int r=NEUT;
17     for(int i0=x0+n,i1=x1+n;i0<i1;i0>>=1,i1>>=1){
18         int t[4],q=0;
19         if(i0&1)t[q++]=i0++;
20         if(i1&1)t[q++]---i1;
21         fore(k,0,q)for(int j0=y0+m,j1=y1+m;j0<j1;j0
            >>=1,j1>>=1){
22             if(j0&1)r=op(r,st[t[k]][j0++]);
23             if(j1&1)r=op(r,st[t[k]][--j1]);
24         }
25     }
26     return r;
27 }
    
```

1.5 Fenwick tree

```

1 int ft[MAXN+1]; // add dimension for multi-d
2 void upd(int i0, int v){ // add v to i0th element
3     for(int i=i0+1;i<=MAXN;i+=i&-i)ft[i]+=v; //+ fors
4 }
5 int get(int i0){ // get sum of range [0,i0]
6     int r=0; // add fors
7     for(int i=i0;i-=i&-i)r+=ft[i];
8     return r;
9 }
10 int get_sum(int i0,int i1){//sum of [i0,i1]
11     return get(i1)-get(i0);
12 }
    
```

1.6 Sparse Table

```

1 #define oper min
2 int st[K][1<<K];int n; // K such that 2^K>n
3 void st_init(int *a){
4     fore(i,0,n)st[0][i]=a[i];
5     fore(k,1,K)fore(i,0,n-(1<<k)+1)
6         st[k][i]=oper(st[k-1][i],st[k-1][i+(1<<(k-1))]);
7 }
8 int st_query(int s, int e){
9     int k=31-__builtin_clz(e-s);
10    return oper(st[k][s],st[k][e-(1<<k)]);
11 }
    
```

1.7 Treap (as BST)

```

1 typedef struct item *pitem;
2 struct item {
3     int pr,key,cnt; pitem l,r;
4     item(int key):key(key),pr(rand()),cnt(1),l(0),r
        (0) {}
5 };
6 int cnt(pitem t){return t?t->cnt:0;}
7 void upd_cnt(pitem t){if(t)t->cnt=cnt(t->l)+cnt(t->
        r)+1;}
8 void split(pitem t, int key, pitem& l, pitem& r){
9     // l: <= key, r: > key
10    if(!t)l=r=0;
11    else if(key<t->key)split(t->l,key,l,t->l),r=t;
12    else split(t->r,key,t->r,r),l=t;
13    upd_cnt(t);
14 }
15 void insert(pitem& t, pitem it){
16     if(!t)t=it;
17     else if(it->pr>t->pr)split(t,it->key,it->l,it->r)
18         ,t=it;
19     else insert(it->key<t->key?t->l:t->r,it);
20     upd_cnt(t);
21 }
22 void merge(pitem& t, pitem l, pitem r){
23     if(!l||!r)t=l?l:r;
24     else if(l->pr>r->pr)merge(l->r,l->r,r),t=l;
25     else merge(r->l,l,r->l),t=r;
26     upd_cnt(t);
27 }
28 void erase(pitem& t, int key){
29     if(t->key==key)merge(t,t->l,t->r);
30     else erase(key<t->key?t->l:t->r,key);
31     upd_cnt(t);
32 }
    
```

1.8 Treap (implicit key)

```

1 typedef struct item *pitem;
2 struct item {
3     int pr,cnt,val; pitem l,r,f;
4     item(int val): pr(rand()),cnt(1),val(val),l(0),r
        (0),f(0){}
5 };
6 void push(pitem it){
7     if(!it)return;
8 }
9 int cnt(pitem t){return t?t->cnt:0;}
10 // int sum(pitem t){return t?push(t),t->sum:0;}
11 void upd_cnt(pitem t){
12     if(t){
    
```

```

13     t->cnt=cnt(t->l)+cnt(t->r)+1;
14     // t->sum=t->val+sum(t->l)+sum(t->r);
15 }
16 }
17 void merge(pitem& t, pitem l, pitem r){
18     push(l);push(r);
19     if(!l||!r)t=l?l:r;
20     else if(l->pr>r->pr)merge(l->r,l->r,r),l->r->f=t=
        l;
21     else merge(r->l,l,r->l),r->l->f=t=r;
22     upd_cnt(t);
23 }
24 void split(pitem t, pitem& l, pitem& r, int sz){
25     if(!t){l=r=0;return;}
26     push(t);
27     if(sz<=cnt(t->l)){
28         split(t->l,l,t->l,sz);r=t;
29         if(l)l->f=0;
30         if(t->l)t->l->f=t;
31     }
32     else {
33         split(t->r,t->r,r,sz-1-cnt(t->l));l=t;
34         if(r)r->f=0;
35         if(t->r)t->r->f=t;
36     }
37     upd_cnt(t);
38 }
39 void push_all(pitem t){
40     if(t->f)push_all(t->f);
41     push(t);
42 }
43 pitem root(pitem t, int& pos){ // get root and
    position for node t
44     push_all(t); pos=cnt(t->l);
45     while(t->f){
46         pitem f=t->f;
47         if(t==f->r)pos+=cnt(f->l)+1;
48         t=f;
49     }
50     return t;
51 }

```

1.9 Link-Cut tree

```

1  const int N_DEL = 0, N_VAL = 0; //delta, value
2  inline int mOp(int x, int y){return x+y;} //modify
3  inline int qOp(int lval, int rval){return lval +
    rval;} //query
4  inline int dOnSeg(int d, int len){return d==N_DEL ?
    N_DEL : d*len;}
5  //mostly generic
6  inline int joinD(int d1, int d2){
7      if(d1==N_DEL)return d2;if(d2==N_DEL)return d1;
8      return mOp(d1, d2);}
9  inline int joinVD(int v, int d){return d==N_DEL ? v
    : mOp(v, d);}
10 struct Node_t{
11     int sz, nVal, tVal, d; bool rev;
12     Node_t *c[2], *p;
13     Node_t(int v) : sz(1), nVal(v), tVal(v), d(N_DEL)
        , rev(0), p(0){
14         c[0]=c[1]=0;
15     }
16     bool isRoot(){return !p || (p->c[0] != this && p

```

```

    ->c[1] != this);}
16 void push(){
17     if(rev){
18         rev=0; swap(c[0], c[1]); fore(x,0,2)if(c[x])c[x]
            ]->rev^=1;
19     }
20     nVal=joinVD(nVal, d); tVal=joinVD(tVal, dOnSeg(d,
        sz));
21     fore(x,0,2)if(c[x])c[x]->d=joinD(c[x]->d, d);
22     d=N_DEL;
23 }
24 void upd();
25 };
26 typedef Node_t* Node;
27 int getSize(Node r){return r ? r->sz : 0;}
28 int getPv(Node r){
29     return r ? joinVD(r->tVal, dOnSeg(r->d,r->sz)) :
        N_VAL;}
30 void Node_t::upd(){
31     tVal = qOp(qOp(getPv(c[0]), joinVD(nVal, d)),
        getPv(c[1]));
32     sz = 1 + getSize(c[0]) + getSize(c[1]);
33 }
34 void conn(Node c, Node p, int il){if(c)c->p=p;if(il
    >=0)p->c[!il]=c;}
35 void rotate(Node x){
36     Node p = x->p, g = p->p;
37     bool gCh=p->isRoot(), isl = x==p->c[0];
38     conn(x->c[isl],p,isl); conn(p,x,!isl);
39     conn(x,g,gCh?-1:(p==g->c[0])); p->upd();
40 }
41 void spa(Node x){//splay
42     while(!x->isRoot()){
43         Node p = x->p, g = p->p;
44         if(!p->isRoot())g->push();
45         p->push(); x->push();
46         if(!p->isRoot())rotate((x==p->c[0])==(p==g->c[0])
            ? p : x);
47         rotate(x);
48     }
49     x->push(); x->upd();
50 }
51 Node exv(Node x){//expose
52     Node last=0;
53     for(Node y=x; y; y=y->p)spa(y),y->c[0]=last,y->
        upd(),last=y;
54     spa(x);
55     return last;
56 }
57 void mkR(Node x){exv(x);x->rev^=1;} //makeRoot
58 Node getR(Node x){exv(x);while(x->c[1])x=x->c[1];
    spa(x);return x;}
59 Node lca(Node x, Node y){exv(x); return exv(y);}
60 bool connected(Node x, Node y){exv(x);exv(y);
    return x==y?1:x->p!=0;}
61 void link(Node x, Node y){mkR(x); x->p=y;}
62 void cut(Node x, Node y){mkR(x); exv(y); y->c[1]->p
    =0; y->c[1]=0;}
63 Node father(Node x){
64     exv(x); Node r=x->c[1];
65     if(!r)return 0;
66     r->push();
67     while(r->c[0])r=r->c[0],r->push();
68     return r;

```

```

69 }
70 void cut(Node x){ // cuts x from father keeping
    tree root
71     exv(father(x));x->p=0;}
72 int query(Node x, Node y){mkR(x); exv(y); return
    getPV(y);}
73 void modify(Node x, Node y, int d){mkR(x);exv(y);y
    ->d=joinD(y->d,d);}
74 Node lift_rec(Node x, int t){
75     if(!x)return 0;
76     if(t==getSize(x->c[0])){spa(x);return x;}
77     if(t<getSize(x->c[0]))return lift_rec(x->c[0],t);
78     return lift_rec(x->c[1],t-getSize(x->c[0])-1);
79 }
80 Node lift(Node x, int t){ // t-th ancestor of x (
    lift(x,1) is x's father)
81     exv(x);return lift_rec(x,t);}
82 int depth(Node x){ // distance from x to its tree
    root
83     exv(x);return getSize(x)-1;}
    
```

1.10 Convex hull trick (static)

```

1 typedef ll tc;
2 struct Line{tc m,h};
3 struct CHT { // for minimum (for maximum just
    change the sign of lines)
4     vector<Line> c; int pos=0;
5     tc in(Line a, Line b){
6         tc x=b.h-a.h,y=a.m-b.m;
7         return x/y+(x%y?!((x>0)^(y>0)):0);//==ceil(x/y)
8     }
9     void add(tc m, tc h){ // m's should be non
        increasing
10        Line l=(Line){m,h};
11        if(c.size()&&m==c.back().m){
12            l.h=min(h,c.back().h);c.pop_back();if(pos)pos
            --;
13        }
14        while(c.size()>1&&in(c.back(),l)<=in(c[c.size()
            -2],c.back())){
15            c.pop_back();if(pos)pos--;
16        }
17        c.pb(l);
18    }
19    inline bool fbin(tc x, int m){return in(c[m],c[m
        +1])>x;}
20    tc eval(tc x){
21        // O(log n) query:
22        int s=0,e=c.size();
23        while(e-s>1){int m=(s+e)/2;
24            if(fbin(x,m-1))e=m;
25            else s=m;
26        }
27        return c[s].m*x+c[s].h;
28        // O(1) query (for ordered x's):
29        while(pos>0&&fbin(x,pos-1))pos--;
30        while(pos<c.size()-1&&!fbin(x,pos))pos++;
31        return c[pos].m*x+c[pos].h;
32    }
33 };
    
```

1.11 Convex hull trick (dynamic)

```

1 typedef ll tc;
    
```

```

2 const tc is_query=-(1LL<<62);//special value for
    query
3 struct Line {
4     tc m,b;
5     mutable multiset<Line>::iterator it,end;
6     const Line* succ(multiset<Line>::iterator it)
        const {
7         return (++it==end? NULL : &*it);}
8     bool operator<(const Line& rhs) const {
9         if(rhs.b!=is_query)return m<rhs.m;
10        const Line *s=succ(it);
11        if(!s)return 0;
12        return b-s->b<(s->m-m)*rhs.m;
13    }
14 };
15 struct HullDynamic: public multiset<Line>{//for MAX
16     bool bad(iterator y){
17         iterator z=next(y);
18         if(y==begin()){
19             if(z==end())return false;
20             return y->m==z->m&&y->b<=z->b;
21         }
22         iterator x=prev(y);
23         if(z==end())return y->m==x->m&&y->b<=x->b;
24         //WARNING: MAY OVERFLOW
25         return (x->b-y->b)*(z->m-y->m)>=(y->b-z->b)*(y
            ->m-x->m);
26     }
27     iterator next(iterator y){return ++y;}
28     iterator prev(iterator y){return --y;}
29     void add(tc m, tc b){
30         iterator y=insert((Line){m,b});
31         y->it=y;y->end=end();
32         if(bad(y)){erase(y);return;}
33         while(next(y)!=end()&&bad(next(y)))erase(next(y
            ));
34         while(y!=begin()&&bad(prev(y)))erase(prev(y));
35     }
36     tc eval(tc x){
37         Line l=*lower_bound((Line){x,is_query}); return
            l.m*x+l.b;
38     }
39 };
    
```

1.12 Li-Chao Tree (dynamic and with merge)

```

1 // LiChaoTree lct(sz) -> allows queries for all x
    in range [0,sz)
2 // lct.add(L) -> add line L to range [0,sz)
    in O(log(sz))
3 // lct.add(L,st,en) -> add line L to range [st,en
    ) in O(log^2(sz))
4 // lct.query(x) -> query maximum L(x) across
    all current lines in O(log(sz))
5 typedef long long T;
6 T INF=1e18;
7 struct Line{
8     T a, b;
9     Line():a(0),b(-INF){}
10    Line(T a, T b):a(a),b(b){}
11    T get(T x){return a*x+b;}
12 };
13 struct LiChaoTree{
14     struct Node{
        
```

```

15   Line line;
16   Node *lc,*rc=0;
17 }*root;
18 int sz;
19 LiChaoTree(int sz): root(new Node()), sz(sz){}
20 void addRec(Node* &n, int tl, int tr, Line x){
21     if(!n) n=new Node();
22     if(n->line.get(tl)<x.get(tl)) swap(n->line,x);
23     if(n->line.get(tr-1)>x.get(tr-1) || tl+1==tr)
24         return;
25     int mid=(tl+tr)/2;
26     if(n->line.get(mid)>x.get(mid)) addRec(n->rc,mid
27         ,tr,x);
28     else {swap(n->line,x); addRec(n->lc,tl,mid,x);}
29 }
30 void add(Node* &n, int tl, int tr, int l, int r,
31     Line x){
32     if(tr<=l || r<=tl) return;
33     if(!n) n=new Node();
34     if(l<=tl&&tr<=r) return addRec(n,tl,tr,x);
35     int mid=(tl+tr)/2;
36     add(n->lc,tl,mid,l,r,x); add(n->rc,mid,tr,l,r,x
37 );
38 }
39 T query(Node* &n, int tl, int tr, int x){
40     if(!n) return -INF;
41     if(tl+1==tr) return n->line.get(x);
42     T res=n->line.get(x);
43     int mid=(tl+tr)/2;
44     if(x<mid) res=max(res, query(n->lc,tl,mid,x));
45     else res=max(res, query(n->rc,mid,tr,x));
46     return res;
47 }
48 void add(Line x){add(root,0,sz,0,sz,x);}
49 void add(Line x, int l, int r){add(root,0,sz,l,r,
50     x);}
51 T query(int x){return query(root,0,sz,x);}
52 };
    
```

2 Graphs

2.1 Dijkstra

```

1   vector<pair<int,int> > g[MAXN]; // u->[(v,cost)]
2   ll dist[MAXN];
3   void dijkstra(int x){
4       memset(dist,-1,sizeof(dist));
5       priority_queue<pair<ll,int> > q;
6       dist[x]=0;q.push({0,x});
7       while(!q.empty()){
8           x=q.top().snd;ll c=-q.top().fst;q.pop();
9           if(dist[x]!=c)continue;
10          for(i,0,g[x].size()){
11              int y=g[x][i].fst; ll c=g[x][i].snd;
12              if(dist[y]<0 || dist[x]+c<dist[y])
13                  dist[y]=dist[x]+c,q.push({-dist[y],y});
14          }
15      }
16  }
    
```

2.2 Floyd Warshall

```

1   // g[i][j]: weight of edge (i, j) or INF if there's
2   // no edge
3   // g[i][i]=0
    
```

```

3   ll g[MAXN][MAXN];int n;
4   void floyd(){ // O(n^3) . Replaces g with min
5       distances
6       for(k,0,n)for(i,0,n)if(g[i][k]<INF)for(j,0,n)
7           if(g[k][j]<INF)
8               g[i][j]=min(g[i][j],g[i][k]+g[k][j]);
9   }
10  bool inNegCycle(int v){return g[v][v]<0;}
11  bool hasNegCycle(int a, int b){ // true iff there's
12      neg cycle in between
13      for(i,0,n)if(g[a][i]<INF&&g[i][b]<INF&&g[i][i]
14          <0)return true;
15      return false;
16  }
    
```

2.3 Kruskal + Union Find

```

1   int uf[MAXN];
2   void uf_init(){memset(uf,-1,sizeof(uf));}
3   int uf_find(int x){return uf[x]<0?x:uf[x]=uf_find(
4       uf[x]);}
5   bool uf_join(int x, int y){
6       x=uf_find(x);y=uf_find(y);
7       if(x==y)return false;
8       if(uf[x]>uf[y])swap(x,y);
9       uf[x]+=uf[y];uf[y]=x;
10      return true;
11  }
12  vector<pair<ll,pair<int,int> > > es; // edges (cost
13      ,(u,v))
14  ll kruskal(){ // assumes graph is connected
15      sort(es.begin(),es.end());uf_init();
16      ll r=0;
17      for(i,0,es.size()){
18          int x=es[i].snd.fst,y=es[i].snd.snd;
19          if(uf_join(x,y))r+=es[i].fst; // (x,y,c)
20          belongs to mst
21      }
22      return r; // total cost
23  }
    
```

2.4 Toposort

```

1   vector<int> g[MAXN];int n;
2   vector<int> tsort(){ // lexicographically smallest
3       topological sort
4       vector<int> r;priority_queue<int> q;
5       vector<int> d(2*n,0);
6       for(i,0,n)for(j,0,g[i].size())d[g[i][j]]++;
7       for(i,0,n)if(!d[i])q.push(-i);
8       while(!q.empty()){
9           int x=-q.top();q.pop();r.pb(x);
10          for(i,0,g[x].size()){
11              d[g[x][i]]--;
12              if(!d[g[x][i]])q.push(-g[x][i]);
13          }
14      }
15      return r; // if not DAG it will have less than n
16      elements
17  }
    
```

2.5 Bellman-Ford

```

1   int n;
2   vector<pair<int,int> > g[MAXN]; // u->[(v,cost)]
    
```



```

3 ll dist[MAXN];
4 void bford(int src){ // 0(nm)
5     fill(dist,dist+n,INF);dist[src]=0;
6     fore(_,0,n)fore(x,0,n)if(dist[x]!=INF)for(auto t:
7         g[x]){
8         dist[t.fst]=min(dist[t.fst],dist[x]+t.snd);
9     }
10    fore(x,0,n)if(dist[x]!=INF)for(auto t:g[x]){
11        if(dist[t.fst]>dist[x]+t.snd){
12            // Reconstruct: save "prev" of each node, go
13            // up from t.fst until repeating a node X.
14            // All nodes between the two occurrences of
15            // X are in the cycle
16        }
17    }
18 }

```

2.6 SCC (+ 2-SAT)

```

1 // Finds strongly connencted components , ord is a
2 // valid toposort of the component graph
3 vv g[MAXN], gt[MAXN], ord; //gt: transposed graph.
4 //ord: order for scc
5 ll vis[MAXN], cmp[MAXN], truth[MAXN], qcmp = 0, n;
6 //cmp[x]:scc of x.truth[x]:bool value for 2sat
7 void dfsk(ll x, ll t){
8     vis[x] = 1; if(t) cmp[x] = qcmp;
9     for(auto y:t ? gt[x]:g[x])if(!vis[y]) dfsk(y,t);
10    if(!t) ord.pb(x);
11 }
12 void scc(){
13     qcmp=0; ord.clear(); mset(vis,0);
14     fore(i,0,n) if(!vis[i]) dfsk(i,0);
15     reverse(ALL(ord)); mset(vis,0);
16     for(auto x:ord)if(!vis[x]) dfsk(x,1),qcmp++;
17 }
18 // 2SAT, first define n = 2 * vars
19 ll neg(ll x){return n-x-1;}
20 void add_or(ll x, ll y){
21     g[neg(x)].pb(y); g[neg(y)].pb(x);
22     gt[x].pb(neg(y)); gt[y].pb(neg(x));
23 }
24 bool satis(){
25     scc();
26     fore(i,0,n){
27         if(cmp[i] == cmp[neg(i)]) return false;
28         truth[i] = cmp[i] > cmp[neg(i)];
29     }
30     return true;
31 }

```

2.7 Articulation - Bridges - Biconnected

```

1 vector<int> g[MAXN];int n;
2 struct edge {int u,v,comp;bool bridge;};
3 vector<edge> e;
4 void add_edge(int u, int v){
5     g[u].pb(e.size());g[v].pb(e.size()); e.pb((edge){
6         u,v,-1,false});
7 }
8 int D[MAXN],B[MAXN],T;
9 int nbc; // number of biconnected components
10 int art[MAXN]; // articulation point iff !=0
11 stack<int> st; // only for biconnected

```

```

11 void dfs(int u,int pe){
12     B[u]=D[u]=T++;
13     for(int ne:g[u])if(ne!=pe){
14         int v=e[ne].u^e[ne].v^u;
15         if(D[v]<0){
16             st.push(ne);dfs(v,ne);
17             if(B[v]>D[u])e[ne].bridge = true; // bridge
18             if(B[v]>=D[u]){
19                 art[u]++; // articulation
20                 int last; // start biconnected
21                 do{ last=st.top();st.pop(); e[last].comp=
22                     nbc; } while(last!=ne);
23                 nbc++; // end biconnected
24             }
25             B[u]=min(B[u],B[v]);
26         }
27         else if(D[v]<D[u])st.push(ne),B[u]=min(B[u],D[v]);
28     }
29 }
30 void doit(){
31     memset(D,-1,sizeof(D));memset(art,0,sizeof(art));
32     nbc=T=0; fore(i,0,n)if(D[i]<0)dfs(i,-1),art[i]--;
33 }

```

2.8 Heavy-Light decomposition

```

1 vector<int> g[MAXN];
2 int wg[MAXN],dad[MAXN],dep[MAXN];
3 void dfs1(int x){
4     wg[x]=1;
5     for(int y:g[x])if(y!=dad[x]){
6         dad[y]=x;dep[y]=dep[x]+1;dfs1(y); wg[x]+=wg[y];
7     }
8 }
9 int curpos,pos[MAXN],head[MAXN];
10 void hld(int x, int c){
11     if(c<0)c=x;
12     pos[x]=curpos++;head[x]=c; int mx=-1;
13     for(int y:g[x])if(y!=dad[x]&&(mx<0||wg[mx]<wg[y]))
14         mx=y;
15     if(mx>=0)hld(mx,c);
16     for(int y:g[x])if(y!=mx&&y!=dad[x])hld(y,-1);
17 }
18 void hld_init(){dad[0]=-1;dep[0]=0;dfs1(0);curpos=
19     0;hld(0,-1);}
20 ll query(int x, int y, STree& rmq){
21     ll r=NEUT;
22     while(head[x]!=head[y]){
23         if(dep[head[x]]>dep[head[y]])swap(x,y);
24         r=oper(r,rmq.query(pos[head[y]],pos[y]+1)); y=
25             dad[head[y]];
26     }
27     if(dep[x]>dep[y])swap(x,y); // now x is lca
28     r=oper(r,rmq.query(pos[x],pos[y]+1));
29     return r;
30 }
31 // rmq.upd(pos[x],v);
32 // edge query: - assign edge-value to "child" node
33 // - change pos[x] to pos[x]+1 in query (line 25)

```

2.9 Centroid decomposition

```

1 // tag[y]>=tag[x] for every y in x's connected
2 // subgraph (unrooted subtree)

```

```

2 vector<int> g[MAXN];
3 int tag[MAXN]; // time of discovery of centroid
4 int fat[MAXN]; // father in centroid decomposition
5 int szt[MAXN]; // size of subtree
6 int calcsz(int x, int f){
7     szt[x]=1;
8     for(auto y:g[x])if(y!=f&&tag[y]<0)szt[x]+=calcsz(
9         y,x);
10    return szt[x];
11 }
12 int ccnt=0;
13 void cdfs(int x, int f, int sz=-1){ // 0(nlogn)
14     if(sz<0)sz=calcsz(x,-1);
15     for(auto y:g[x])if(tag[y]<0&&szt[y]*2>=sz){
16         szt[x]=0;cdfs(y,f,sz);return;
17     }
18     tag[x]=ccnt++; fat[x]=f;
19     for(auto y:g[x])if(tag[y]<0)cdfs(y,x);
20 }
21 void centroid(){memset(tag,-1);ccnt=0;cdfs(0,-1);}

```

2.10 Parallel DFS

```

1 struct Tree {
2     int n,z[2];
3     vector<vector<int>> g;
4     vector<int> ex,ey,p,w,f,v[2];
5     Tree(int n):g(n),w(n),f(n){}
6     void add_edge(int x, int y){
7         p.pb(g[x].size());g[x].pb(ex.size());ex.pb(x);
8         ey.pb(y);
9         p.pb(g[y].size());g[y].pb(ex.size());ex.pb(y);
10        ey.pb(x);
11    }
12    bool go(int k){//returns 1 if it finds new node
13        int& x=z[k];
14        while(x>=0&&
15            (w[x]==g[x].size()||w[x]==g[x].size()-1&&(g[x]
16                .back()^1)==f[x]))
17            x=f[x]>=0?ex[f[x]]:-1;
18        if(x<0)return false;
19        if((g[x][w[x]]^1)==f[x])w[x]++;
20        int e=g[x][w[x]],y=ey[e]; f[y]=e; w[x]++; w[y]
21        ]=0; x=y; v[k].pb(x);
22        return true;
23    }
24    vector<int> erase_edge(int e){
25        e*=2;//erases eth edge, returns smaller comp
26        int x=ex[e],y=ey[e]; p[g[x].back()]=p[e];
27        g[x][p[e]]=g[x].back(); g[x].pop_back();
28        p[g[y].back()]=p[e^1]; g[y][p[e^1]]=g[y].back()
29        ; g[y].pop_back();
30        f[x]=f[y]=-1; w[x]=w[y]=0; z[0]=x;z[1]=y; v
31        [0]={x};v[1]={y};
32        bool d0=true,d1=true; while(d0&&d1)d0=go(0),d1=
33        go(1);
34        return v[1-d1];
35    }
36 }

```

2.11 Eulerian path

```

1 // Directed version (uncomment commented code for
2 // undirected)
3 struct edge {

```

```

3     int y; // list<edge>::iterator rev;
4     edge(int y):y(y){}
5 };
6 list<edge> g[MAXN];
7 void add_edge(int a, int b){
8     g[a].push_front(edge(b));//auto ia=g[a].begin();
9     // g[b].push_front(edge(a));auto ib=g[b].begin();
10    // ia->rev=ib;ib->rev=ia;
11 }
12 vector<int> p;
13 void go(int x){
14     while(g[x].size()){
15         int y=g[x].front().y; //g[y].erase(g[x].front()
16             .rev);
17         g[x].pop_front(); go(y);
18     }
19     p.push_back(x);
20 }
21 vector<int> get_path(int x){ // get a path that
22     begins in x
23     // check that a path exists from x before calling
24     to get_path!
25     p.clear();go(x);reverse(p.begin(),p.end());
26     return p;
27 }

```

2.12 Union-Find Rollback

```

1 struct UnionFind {
2     int n,comp;
3     vector<int> uf,si,c;
4     UnionFind(int n=0):n(n),comp(n),uf(n),si(n,1){
5         fore(i,0,n)uf[i]=i; }
6     int find(int x){return x==uf[x]?x:find(uf[x]);}
7     bool join(int x, int y){
8         if((x=find(x))==y)return false;
9         if(si[x]<si[y])swap(x,y);
10        si[x]+=si[y];uf[y]=x;comp--;c.pb(y);
11        return true;
12    }
13    int snap(){return c.size();}
14    void rollback(int snap){
15        while(c.size()>snap){
16            int x=c.back();c.pop_back(); si[uf[x]]-=si[x]
17            ; uf[x]=x;comp++;
18        }
19    }
20 }

```

2.13 Dynamic connectivity

```

1 //need rollback UF
2 enum {ADD,DEL,QUERY};
3 struct Query {int type,x,y};
4 struct DynCon {
5     vector<Query> q; UnionFind dsu; vector<int> mt;
6     map<pair<int,int>,int> last;
7     DynCon(int n):dsu(n){}
8     void add(int x, int y){
9         if(x>y)swap(x,y);
10        q.pb((Query){ADD,x,y});mt.pb(-1);last[{x,y}]=q.
11        size()-1;
12    }
13    void remove(int x, int y){
14        if(x>y)swap(x,y);

```

```

14     q.pb((Query){DEL,x,y});
15     int pr=last[{x,y}];mt[pr]=q.size()-1;mt.pb(pr);
16 }
17 void query(){q.pb((Query){QUERY,-1,-1});mt.pb(-1);
18 }
19 void process(){ // answers all queries in order
20     if(!q.size())return;
21     fore(i,0,q.size())if(q[i].type==ADD&&mt[i]<0)mt
22         [i]=q.size();
23     go(0,q.size());
24 }
25 void go(int s, int e){
26     if(s+1==e){
27         if(q[s].type==QUERY)//answer query using DSU
28             printf("%d\n",dsu.comp);
29         return;
30     }
31     int k=dsu.snap(),m=(s+e)/2;
32     for(int i=e-1;i>=m;--i)if(mt[i]>=0&&mt[i]<s)dsu
33         .join(q[i].x,q[i].y);
34     go(s,m);dsu.rollback(k);
35     for(int i=m-1;i>=s;--i)if(mt[i]>=e)dsu.join(q[i]
36         ].x,q[i].y);
37     go(m,e);dsu.rollback(k);
38 }
39 };

```

2.14 Directed MST (minimum spanning arborescence)

```

1 // need rollbackUF
2 struct Edge{int a, b; ll w;};
3 struct Node{
4     Edge key;
5     Node *l, *r;
6     ll delta;
7     void prop(){
8         key.w += delta;
9         if(l) l->delta += delta;
10        if(r) r->delta += delta;
11        delta = 0;
12    }
13    Edge top(){prop(); return key;}
14 };
15 Node *merge(Node *a, Node *b){
16     if(!a || !b) return a ? b;
17     a->prop(), b->prop();
18     if(a->key.w > b->key.w) swap(a,b);
19     swap(a->l,(a->r = merge(b,a->r)));
20     return a;
21 }
22 void pop(Node *&a){a->prop(); a = merge(a->l,a->r);
23 }
24 pair<ll,vv> dmst(int n, int r, vector<Edge>&g){
25     UnionFind uf(n);
26     vector<Node*> heap(n);
27     for(Edge e:g) heap[e.b] = merge(heap[e.b], new
28         Node{e});
29     ll res = 0;
30     vv seen(n,-1), path(n), par(n);
31     seen[r] = r;
32     vector<Edge> Q(n), in(n,{ -1,-1}), comp;
33     deque<tuple<int, int, vector<Edge>>> cys;
34     fore(s,0,n){
35         int u = s, qi = 0, w;

```

```

36         while(seen[u] < 0){
37             if(!heap[u]) return {-1,{}};
38             Edge e = heap[u] -> top();
39             heap[u] -> delta -= e.w, pop(heap[u]);
40             Q[qi] = e, path[qi++] = u, seen[u] = s
41             ;
42             res += e.w, u = uf.find(e.a);
43             if(seen[u] == s){
44                 Node *cyc = 0;
45                 int end = qi, time = uf.snap();
46                 do cyc = merge(cyc, heap[w = path
47                     [--qi]]);
48                 while(uf.join(u,w));
49                 u = uf.find(u), heap[u] = cyc, seen
50                     [u] = -1;
51                 cys.push_front({u,time,{&Q[qi], &Q
52                     [end]}});
53             }
54         }
55         fore(i,0,qi)in[uf.find(Q[i].b)] = Q[i];
56     }
57     for(auto & [u,t,comp] : cys){
58         uf.rollback(t);
59         Edge inEdge = in[u];
60         for(auto &e : comp) in[uf.find(e.b)] = e;
61         in[uf.find(inEdge.b)] = inEdge;
62     }
63     fore(i,0,n) par[i] = in[i].a;
64     return {res,par};
65 }

```

2.15 Edmond's blossom (matching in general graphs)

```

1 vector<int> g[MAXN];
2 int n,m,mt[MAXN],qh,qt,q[MAXN],ft[MAXN],bs[MAXN];
3 bool inq[MAXN],inb[MAXN],inp[MAXN];
4 int lca(int root, int x, int y){
5     memset(inp,0,sizeof(inp));
6     while(1){
7         inp[x=bs[x]]=true; if(x==root)break;
8         x=ft[mt[x]];
9     }
10    while(1){
11        if(inp[y=bs[y]])return y;
12        else y=ft[mt[y]];
13    }
14 }
15 void mark(int z, int x){
16     while(bs[x]!=z){
17         int y=mt[x];inb[bs[x]]=inb[bs[y]]=true;x=ft[y];
18         if(bs[x]!=z)ft[x]=y;
19     }
20 }
21 void contr(int s, int x, int y){
22     int z=lca(s,x,y);
23     memset(inb,0,sizeof(inb));
24     mark(z,x);mark(z,y);
25     if(bs[x]!=z)ft[x]=y;
26     if(bs[y]!=z)ft[y]=x;
27     fore(x,0,n)if(inb[bs[x]]){
28         bs[x]=z; if(!inq[x])inq[q[++qt]=x]=true;
29     }
30 }

```



```

31 int findp(int s){
32     memset(inq,0,sizeof(inq));
33     memset(ft,-1,sizeof(ft));
34     fore(i,0,n)bs[i]=i;
35     inq[q[qh=qt=0]=s]=true;
36     while(qh<=qt){
37         int x=q[qh++];
38         for(int y:g[x])if(bs[x]!=bs[y]&&mt[x]!=y){
39             if(y==s|mt[y]>=0&&ft[mt[y]]>=0)contr(s,x,y);
40             else if(ft[y]<0){
41                 ft[y]=x;
42                 if(mt[y]<0)return y;
43                 else if(!inq[mt[y]])inq[q[++qt]=mt[y]]=true;
44             }
45         }
46     }
47     return -1;
48 }
49 int aug(int s, int t){
50     int x=t,y,z;
51     while(x>0)y=ft[x],z=mt[y],mt[y]=x,mt[x]=y,x=z;
52     return t>=0;
53 }
54 int edmonds(){ // O(n^2 m)
55     int r=0; memset(mt,-1,sizeof(mt));
56     fore(x,0,n)if(mt[x]<0)r+=aug(x,findp(x));
57     return r;
58 }

```

2.16 Dominator tree

```

1 //idom[i]=par of i in dominator tree with root=rt,
  // or -1 if not exists
2 int n,rnk[MAXN],pre[MAXN],anc[MAXN],idom[MAXN],semi
  [MAXN],low[MAXN];
3 vector<int> g[MAXN],rev[MAXN],dom[MAXN],ord;
4 void dfspre(int pos){
5     rnk[pos]=SZ(ord); ord.pb(pos);
6     for(auto x:g[pos]){
7         rev[x].pb(pos);
8         if(rnk[x]==n) pre[x]=pos,dfspre(x);
9     }
10 }
11 int eval(int v){
12     if(anc[v]<n&&anc[anc[v]]<n){
13         int x=eval(anc[v]);
14         if(rnk[semi[low[v]]]>rnk[semi[x]]) low[v]=x;
15         anc[v]=anc[anc[v]];
16     }
17     return low[v];
18 }
19 void dominators(int rt){
20     fore(i,0,n){
21         dom[i].clear(); rev[i].clear();
22         rnk[i]=pre[i]=anc[i]=idom[i]=n;
23         semi[i]=low[i]=i;
24     }
25     ord.clear(); dfspre(rt);
26     for(int i=SZ(ord)-1;i--){
27         int w=ord[i];
28         for(int v:rev[w]){
29             int u=eval(v);
30             if(rnk[semi[w]]>rnk[semi[u]])semi[w]=semi[u];

```

```

31     }
32     dom[semi[w]].pb(w); anc[w]=pre[w];
33     for(int v:dom[pre[w]]){
34         int u=eval(v); idom[v]=(rnk[pre[w]]>rnk[semi[
35             u])?u:pre[w];
36     }
37     dom[pre[w]].clear();
38 }
39 for(int w:ord) if(w!=rt&&idom[w]!=semi[w]) idom[w
40     ]=idom[idom[w]];
41 fore(i,0,n) if(idom[i]==n)idom[i]=-1;
42 }

```

2.17 Rerooting DP

```

1 vector<ii> g[MAXN]; // node, weight (1 if none)
2
3 // to see what these mean, look at dfs1
4 typedef ll node;
5 node NEUT;
6 node leaf(ll x){return NEUT;}
7 node merge(node a, node b){}
8 node up(node x, ll w){}
9
10 node h[MAXN],ch[MAXN]; // hijo, complement hijo (
11     SIN ARISTA PADRE)
12 node tot[MAXN]; // total
13 ll wf[MAXN],fa[MAXN]; // weight father, father
14 void dfs1(ll x){
15     h[x]=leaf(x);
16     for(auto [y,w]:g[x])if(y!=fa[x]){
17         wf[y]=w; fa[y]=x; dfs1(y);
18         h[x]=merge(h[x],up(h[y],w));
19     }
20 vector<node> pre[MAXN],suf[MAXN];
21 void dfs2(ll x){
22     fore(j,0,SZ(g[x])){
23         auto [y,w]=g[x][j];
24         if(y==fa[x])continue;
25         ch[y]=merge(leaf(x),merge(pre[x][j],suf[x][j
26             +1]));
27         if(fa[x]!=-1)ch[y]=merge(ch[y],up(ch[x],wf[x]));
28     }
29     dfs2(y);
30 }
31 void reroot(ll n){
32     ll rt=0; fa[rt]=-1,wf[rt]=0; dfs1(rt);
33     fore(x,0,n){
34         fore(ty,0,2){
35             auto &p=(ty?suf[x]:pre[x]);
36             p=vector<node>(SZ(g[x])+1,NEUT);
37             fore(j,1,SZ(p)){
38                 auto [y,w]=g[x][j-1]; p[j]=p[j-1];
39                 if(y!=fa[x])p[j]=merge(p[j],up(h[y],w));
40             }
41             reverse(ALL(g[x]));
42         }
43         reverse(ALL(suf[x]));
44     }
45     dfs2(rt);
46     fore(x,0,n){
47         tot[x]=h[x];

```

```

47     if(fa[x]!=-1)tot[x]=merge(tot[x],up(ch[x],wf[x
48         ]));
49     // you may want to treat the root specially
50     // if that is the case, remember to recalculate
51     tot[rt]
    }
}

```

2.18 Virtual Trees

```

1 // assumes sorted v (in dfs order)
2 // virtual (directed) tree is t
3 void agr(ll x, ll y){if(y!=-1)t[x].pb(y);}
4 ll virtu(vv v){ // 0(|v|) * 0(lca)
5     stack<ll>s; s.push(v[0]); ll ult=-1,p;
6     auto vacia=[&](bool fg){
7         while(SZ(s)&&(fg||lca(s.top(),p)!=s.top())){
8             agr(s.top(),ult);
9             ult=s.top(); s.pop();
10        }
11    };
12    vv vi; // virtual nodes and possibly normal
13    fore(i,1,SZ(v)){
14        ll x=v[i];p=lca(s.top(),x); vi.pb(p); vacia(0);
15        if(s.empty()||p!=s.top())s.push(p);
16        agr(p,ult); ult=-1; if(p!=x)s.push(x);
17    }
18    vacia(1); ll rt=ult; // root of t
19    // do stuff, then reset t with both v and vi
20 }

```

2.19 Tree DSU

```

1 vi g[N]; int sz[N]; bool bigboy[N];
2 void add(int x, int y = -1, int k) {
3     for(auto v:g[x])if(v!=y&&!bigboy[v])add(v,x,k);
4     // do something
5 }
6 void dfs(int x,int y=-1,int k=1){//dont forget prec
7     int mx = -1, h = -1;
8     for(auto v:g[x])if(v!=y&&mx<sz[v])mx=sz[v],h=v;
9     for(auto v:g[x])if(v!=y&&v!=h)dfs(v,x,0);
10    if(h != -1) bigboy[h] = 1, dfs(h, x, 1);
11    add(x, y, 1); solve(x, y);
12    if(h != -1) bigboy[h] = 0;
13    if(!k) add(x, y, -1);
14 }

```

2.20 Edge coloring bipartite

```

1 struct EC{ // for bipartite graphs 0(n + m)
2     vector<ii> has[2] [MAXN]; // unordered_map if
3     MAX_DEGREE is large
4     int deg[2] [MAXN],col [MAXM],c[2];
5     void dfs(int x, int p){
6         auto i=has[p][x][c[!p]];
7         if(has[!p][i.fst][c[p]].snd)dfs(i.fst,!p);
8         else has[!p][i.fst][c[!p]]={0,0};
9         has[p][x][c[p]]=i,has[!p][i.fst][c[p]]={x,i.snd};
10        col[i.snd]=c[p];
11    }
12    int solve(vector<ii> &e, vector<int> &res){ //
13        edges (x, y), result vector
14        int m=SZ(e),ans=0;
15        fore(i,0,m)ans=max({ans,++deg[0][e[i].fst+1],++
16            deg[1][e[i].snd+1]});

```

```

14     fore(i,0,2)fore(j,0,MAXN)has[i][j].resize(ans
15         +1,{0,0});
16     fore(i,0,m){
17         int x[2]={e[i].fst+1,e[i].snd+1};
18         fore(d,0,2)for(c[d]=1;has[d][x[d]][c[d]].snd;c[
19             d]++);
20         if(c[0]!=c[1])dfs(x[1],1);
21         fore(d,0,2)has[d][x[d]][c[0]]={x[!d],i+1};
22         col[i+1]=c[0];
23     }
24     res.resize(m);fore(i,0,m)res[i]=col[i+1]; //
25     color of each edge
26     return ans; // amount of colors used
27 }
} ec;

```

3 Math

3.1 Pollard's rho

```

1 //cb6b1e
2 ll mulmod(ll a, ll b, ll m) {
3     ll r=a*b-(ll)((long double)a*b/m+.5)*m;
4     return r<0?r+m:r;
5 }
6 bool is_prime_prob(ll n, int a){
7     if(n==a)return true;
8     ll s=0,d=n-1;
9     while(d%2==0)s++,d/=2;
10    ll x=expmod(a,d,n);
11    if((x==1)|| (x+1==n))return true;
12    fore(_,0,s-1){
13        x=mulmod(x,x,n);
14        if(x==1)return false;
15        if(x+1==n)return true;
16    }
17    return false;
18 }
19 bool rabin(ll n){ // true iff n is prime
20     if(n==1)return false;
21     int ar[]={2,3,5,7,11,13,17,19,23};
22     fore(i,0,9)if(!is_prime_prob(n,ar[i]))return
23         false;
24     return true;
25 }
26 ll rho(ll n){
27     if(!(n&1))return 2;
28     ll x=2,y=2,d=1;
29     ll c=rand()%n+1;
30     while(d==1){
31         x=(mulmod(x,x,n)+c)%n;
32         fore(it,0,2) y=(mulmod(y,y,n)+c)%n;
33         if(x>y)d=__gcd(x-y,n);
34         else d=__gcd(y-x,n);
35     }
36     return d==n?rho(n):d;
37 }
38 void fact(ll n, map<ll,int>& f){ //0 (lg n)^3
39     if(n==1)return;
40     if(rabin(n)){f[n]++;return;}
41     ll q=rho(n);fact(q,f);fact(n/q,f);
42 }
43 // optimized version: replace rho and fact with the
44 following:
45 const int MAXP=1e6+1; // sieve size

```

```

44 int sv[MAXP]; // sieve
45 ll add(ll a, ll b, ll m){return (a+=b)<m?a:a-m;}
46 ll rho(ll n){
47     static ll s[MAXP];
48     while(1){
49         ll x=rand()%n,y=x,c=rand()%n;
50         ll *px=s,*py=s,v=0,p=1;
51         while(1){
52             *py++=y=add(mulmod(y,y,n),c,n);
53             *py++=y=add(mulmod(y,y,n),c,n);
54             if((x=*px++)==y)break;
55             ll t=p; p=mulmod(p,abs(y-x),n);
56             if(!p)return __gcd(t,n);
57             if(++v==26){
58                 if((p=__gcd(p,n))>1&&p<n)return p;
59                 v=0;
60             }
61         }
62         if(v&&(p=__gcd(p,n))>1&&p<n)return p;
63     }
64 }
65 void init_sv(){ fore(i,2,MAXP)if(!sv[i])for(ll j=i;
66     j<MAXP;j+=i)sv[j]=i; }
67 void fact(ll n,map<ll,int>&f){//call init_sv first!
68     for(auto&& p:f)while(n%p.fst==0)p.snd++,n/=p.fst;
69     if(n<MAXP)while(n>1)f[sv[n]]++,n/=sv[n];
70     else if(rabin(n))f[n]++;
71     else {ll q=rho(n);fact(q,f);fact(n/q,f);}
72 }

```

3.2 Simpson's rule

```

1 double integrate(double f(double), double a, double
    b, int n=10000){
2     double r=0,h=(b-a)/n,fa=f(a),fb;
3     fore(i,0,n){fb=f(a+h*(i+1));r+=fa+4*f(a+h*(i+0.5)
4         )+fb;fa=fb;}
5     return r*h/6.;
6 }

```

3.3 Polynomials

```

1 typedef int tp; // type of polynomial
2 template<class T=tp>
3 struct poly { // poly<> : 1 variable, poly<poly
4     <>>: 2 variables, etc.
5     vector<T> c;
6     T& operator[](int k){return c[k];}
7     poly(vector<T>& c):c(c){}
8     poly(initializer_list<T> c):c(c){}
9     poly(int k):c(k){}
10    poly(){}
11    poly operator+(poly<T> o);
12    poly operator*(tp k);
13    poly operator*(poly o);
14    poly operator-(poly<T> o){return *this+(o*-1);}
15    T operator()(tp v){
16        T sum(0);
17        for(int i=c.size()-1;i>=0;--i)sum=sum*v+c[i];
18        return sum;
19    }
20    // example: p(x,y)=2*x^2+3*x*y-y+4
21    // poly<poly<>> p={{4,-1},{0,3},{2}}
22    // printf("%d\n",p(2)(3)) // 27 (p(2,3))

```

```

23 set<tp> roots(poly<> p){ // only for integer
    polynomials
24     set<tp> r;
25     while(!p.c.empty()&&!p.c.back())p.c.pop_back();
26     if(!p(0))r.insert(0);
27     if(p.c.empty())return r;
28     tp a0=0,an=abs(p[p.c.size()-1]);
29     for(int k=0;!a0;a0=abs(p[k++]));
30     vector<tp> ps,qs;
31     fore(i,1,sqrt(a0)+1)if(a0%i==0)ps.pb(i),ps.pb(a0/i);
32     fore(i,1,sqrt(an)+1)if(an%i==0)qs.pb(i),qs.pb(an/i);
33     for(auto pt:ps)for(auto qt:qs)if(pt%qt==0){
34         tp x=pt/qt;
35         if(!p(x))r.insert(x);
36         if(!p(-x))r.insert(-x);
37     }
38     return r;
39 }
40 pair<poly<>,tp> ruffini(poly<> p, tp r){ // returns
    pair (result,rem)
41     int n=p.c.size()-1;
42     vector<tp> b(n);
43     b[n-1]=p[n];
44     for(int k=n-2;k>=0;--k)b[k]=p[k+1]+r*b[k+1];
45     return {poly<>(b),p[0]+r*b[0]};
46 }
47 // only for double polynomials
48 pair<poly<>,poly<>> polydiv(poly<> p, poly<> q){
    // returns pair (result,rem)
49     int n=p.c.size()-q.c.size()+1;
50     vector<tp> b(n);
51     for(int k=n-1;k>=0;--k){
52         b[k]=p.c.back()/q.c.back();
53         fore(i,0,q.c.size())p[i+k]-=b[k]*q[i];
54         p.c.pop_back();
55     }
56     while(!p.c.empty()&&abs(p.c.back())<EPS)p.c.
57         pop_back();
58     return {poly<>(b),p};
59 }
60 // only for double polynomials
61 poly<> interpolate(vector<tp> x, vector<tp> y){
62     poly<> q={1},S={0};
63     for(tp a:x)q=poly<>({-a,1})*q;
64     fore(i,0,x.size()){
65         poly<> Li=ruffini(q,x[i]).fst;
66         Li=Li*(1.0/Li(x[i])); // change for int
67         polynomials
68         S=S+Li*y[i];
69     }
70     return S;
71 }

```

3.4 Number Theoretic Transform

```

1 // d35827
2 // The maximum length of the resulting convolution
    vector is 2^LG
3 const int LG = 20;
4 typedef vv poly;
5
6 template<class u, class uu, u p, u root>

```

```

7 struct FFT {
8     u r[1+(2<<LG)];
9     constexpr u m(u a, u b) {
10         uu k = uu(a)*b;
11         #define op(g) g*(g*p+2)
12         k += u(k) * (op(op(op(op(op(-p)))))) * uu(p);
13         #undef op
14         return u(k)>>(8*sizeof(u));
15     }
16     constexpr u red(u k, u a){return a-k*(a>=k);}
17     FFT() {
18         u k=r[2<<LG]=-p/p, b=root, e=p>>LG;
19         for(;e/=2,b=m(b,b)) if(e&1) k=m(k,b);
20         for(int i=(2<<LG)-1;i>=0;i--) r[i]=red(p, m(r[i
21             +1], k)), i&(i-1)?0:k=m(k,k);
22     }
23     poly cv(const poly &as, const poly &bs, u *v) {
24         int c=max(SZ(as)+SZ(bs)-1, 0ll), n=1;
25         assert(c <= (1<<LG));
26         u h=u(uu(-p)*-p/p), a=m(h, p/2+1), x, y;
27         while(n<c) n*=2, h=red(p, m(h, a));
28         fore(i,0,n){
29             v[i]=i<SZ(as)?u(as[i]):0,
30             v[i+n]=i<SZ(bs)?u(bs[i]):0;
31         }
32         for(auto s:{v,v+n}) for(int j=n;j>=2;j--) for(
33             int k=j&-j; k/=2;) fore(i,j-k,j){
34             x=s[i], y=s[i-k];
35             s[i-k] = red(2*p, x+y);
36             s[i] = m(2*p+y-x, r[3*k-j+i]);
37         }
38         fore(i,0,n) v[i]=m(v[i], v[i+n]);
39         fore(j,2,n+1) for(int k=1; !(k&j); k*=2) fore(i
40             ,j-k,j){
41             x = m(v[i], r[3*k+j-i]);
42             y = red(2*p, v[i-k]);
43             v[i-k]=x+y, v[i]=2*p+y-x;
44         }
45         fore(i,0,c) v[i]=red(p, m(v[i], h));
46         return poly(v, v+c);
47     }
48     // Replace with any NTT-friendly mod by doing:
49     // FFT<uint32_t, uint64_t, MOD, primitiveRoot(MOD)>
50     poly conv_small(const poly &as, const poly &bs) {
51         static uint32_t v[2<<LG];
52         static FFT<uint32_t, uint64_t, 998244353, 3> fft;
53         return fft.cv(as, bs, v);
54     }
55     // For modular convolutions modulo a 62 bit prime.
56     poly conv_big(const poly &as, const poly &bs) {
57         static uint64_t v[2<<LG];
58         static FFT<uint64_t, __uint128_t, (1ull<<62)-(18
59             ull<<32)+1, 3> fft;
60         return fft.cv(as, bs, v);
61     }
62     // For modular convolutions modulo an arbitrary 32-
63     // bit modulus.
64     // Slightly slower than above versions, because it
65     // performs two calls to fft

```

```

65 poly conv_sunzi(const poly &v1, const poly &v2, ll
66     m) {
67     const uint64_t inv = 2703402103339935109ull,
68     mod1 = (1ull<<62)-(18ull<<32)+1,
69     mod2 = (1ull<<62)-(76ull<<32)+1;
70     static uint64_t v[2<<LG];
71     static FFT<uint64_t, __uint128_t, mod1, 3> fft1;
72     static FFT<uint64_t, __uint128_t, mod2, 17> fft2;
73     auto as=fft1.cv(v1, v2, v), bs=fft2.cv(v1, v2, v)
74     ;
75     fore(i,0,SZ(as)){
76         auto d = fft1.m(mod1+as[i]-bs[i], inv);
77         d -= mod1*(d >= mod1); d %= m;
78         as[i] = (bs[i] + mod2/m*d)%m;
79     }
80     return as;
81 }

```

3.5 Fast Fourier Transform

```

1 // MAXN must be power of 2 !!
2 // MOD-1 needs to be a multiple of MAXN !!
3 CD cp1[MAXN+9],cp2[MAXN+9]; int R[MAXN+9];
4 void dft(CD* a, int n, bool inv){
5     fore(i,0,n)if(R[i]<i)swap(a[R[i]],a[i]);
6     for(int m=2;m<=n;m*=2){
7         double z=2*pi/m*(inv?-1:1); CD wi=CD(cos(z),sin
8             (z));
9         for(int j=0;j<n;j+=m){
10             CD w(1);
11             for(int k=j,k2=j+m/2;k2<j+m;k++,k2++){
12                 CD u=a[k];CD v=a[k2]*w;a[k]=u+v;a[k2]=u-v;w
13                     =w*wi;
14             }
15         }
16         if(inv)fore(i,0,n)a[i]/=n;
17     }
18     poly multiply(poly& p1, poly& p2){
19         int n=p1.size()+p2.size()+1,m=1,cnt=0;
20         while(m<=n)m+=m,cnt++;
21         fore(i,0,m){R[i]=0;fore(j,0,cnt)R[i]=(R[i]<<1)|((
22             i>>j)&1);}
23         fore(i,0,m)cp1[i]=0,cp2[i]=0;
24         fore(i,0,p1.size())cp1[i]=p1[i];
25         fore(i,0,p2.size())cp2[i]=p2[i];
26         dft(cp1,m,false);dft(cp2,m,false);
27         fore(i,0,m)cp1[i]=cp1[i]*cp2[i];
28         dft(cp1,m,true); poly res; n-=2;
29         fore(i,0,n)res.pb((tf)floor(cp1[i].real()+0.5));
30         return res;
31     }
32 }

```

3.6 FFT operations

```

1 // Polynomial division:  $O(n \log(n))$ 
2 // Multi-point polynomial evaluation for
3 // arbitrarily large points:  $O(n+k \log^2(k))$ 
4 // Multi-point polynomial evaluation for all points
5 // in  $[0, \text{MOD})$ :  $O((n+\text{MOD}) \log(n+\text{MOD}))$ 
6 // Polynomial interpolation:  $O(n \log^2(n))$ 
7 // Inverse:  $O(n \log(n))$ 
8 // Log:  $O(n \log(n))$ 
9 // Exp:  $O(n \log(n))$ 

```

```

9 //Works with NTT. For FFT, just replace addmod,
  submod,mulmod,inv
10 poly add(const poly &a, const poly &b){
11     int n=SZ(a),m=SZ(b);
12     poly ans(max(n,m));
13     fore(i,0,max(n,m)) ans[i]=addmod(i<SZ(a)?a[i]:0,
        i<SZ(b)?b[i]:0);
14     while(SZ(ans)>1&&!ans.back())ans.pop_back();
15     return ans;
16 }
17
18 // derivative of p
19 poly derivative(const poly &p){
20     poly ans(max(1, SZ(p)-1));
21     fore(i,1,SZ(p)) ans[i-1]=mulmod(p[i],i);
22     return ans;
23 }
24
25 // integral of p
26 poly integrate(const poly &p){
27     int n=SZ(p);
28     poly ans(n+1);
29     if(!n) return ans;
30     poly pre(n+1); pre[0]=1;
31     fore(i,1,n+1) pre[i]=mulmod(pre[i-1],i);
32     tf iv=inv(pre[n]);
33     for(int i=n;i>=1;i--){
34         ans[i]=mulmod(p[i-1], mulmod(iv,pre[i-1]));
35         iv=mulmod(iv,i);
36     }
37     return ans;
38 }
39
40 // p % (x^n)
41 poly takemod(const poly &p, int n){
42     poly res=p;
43     res.resize(min(SZ(res),n));
44     while(SZ(res)>1&&res.back()==0) res.pop_back();
45     return res;
46 }
47
48 // first d terms of 1/p
49 poly invert(const poly &p, int d){
50     assert(p[0]);
51     poly res={inv(p[0])};
52     int sz=1;
53     while(sz<d){
54         sz*=2;
55         poly pre(p.begin(), p.begin()+min(SZ(p),sz));
56         poly cur=multiply(res,pre);
57         fore(i,0,SZ(cur)) cur[i]=submod(0,cur[i]);
58         cur[0]=addmod(cur[0],2);
59         cur=takemod(cur,sz);
60         res=multiply(res,cur);
61         res=takemod(res,sz);
62     }
63     res.resize(d);
64     return res;
65 }
66
67 // first d terms of log(p)
68 poly log(const poly &p, int d){
69     assert(p[0]==1);
70     poly cur=takemod(p,d);

```

```

71     poly a=invert(cur,d), b=derivative(cur);
72     auto res=multiply(a,b);
73     res=takemod(res,d-1);
74     res=integrate(res);
75     return res;
76 }
77
78 // first d terms of exp(p)
79 poly exp(const poly &p, int d){
80     assert(!p[0]);
81     poly res={1};
82     int sz=1;
83     while(sz<d){
84         sz*=2;
85         poly lg=log(res, sz), cur(sz);
86         fore(i,0,sz) cur[i]=submod(i<SZ(p)?p[i]:0, i<SZ
            (lg)?lg[i]:0);
87         cur[0]=addmod(cur[0],1);
88         res=multiply(res,cur);
89         res=takemod(res, sz);
90     }
91
92     res.resize(d);
93     return res;
94 }
95
96 pair<poly,poly> divslow(const poly &a, const poly &
    b){
97     poly q,r=a;
98     tf ib=inv(b.back());
99     while(SZ(r)>=SZ(b)){
100         q.pb(mulmod(r.back(),ib));
101         if(q.back()) fore(i,0,SZ(b)){
102             r[SZ(r)-i-1]=submod(r[SZ(r)-i-1],mulmod(q.
                back(),b[SZ(b)-i-1]));
103         }
104         r.pop_back();
105     }
106     reverse(ALL(q));
107     return {q,r};
108 }
109
110 pair<poly,poly> divide(const poly &a, const poly &b
    ){ //returns {quotient,remainder}
111     int m=SZ(a),n=SZ(b),MAGIC=128;
112     if(m<n) return {{0},a};
113     if(min(m-n,n)<MAGIC)return divslow(a,b);
114     poly ap=a; reverse(ALL(ap));
115     poly bp=b; reverse(ALL(bp));
116     bp=invert(bp,m-n+1);
117     poly q=multiply(ap,bp);
118     q.resize(m-n+1,0);
119     reverse(ALL(q));
120     poly bq=multiply(b,q);
121     fore(i,0,SZ(bq)) bq[i]=submod(0,bq[i]);
122     poly r=add(a,bq);
123     return {q,r};
124 }
125
126 struct EvalTree {
127     int k;
128     vector<poly> tree;
129     EvalTree(const vector<tf> &x): k(SZ(x)), tree(2*
        max(SZ(x),1)) {

```



```

130     fore(i,k,2*k) tree[i]={submod(0,x[i-k]),1};
131     for(int i=k-1;i;i--) tree[i]=multiply(tree[2*i
132         ],tree[2*i+1]);
133 }
134 };
135 poly mult(const poly &M, const poly &a, int n){
136     poly c=multiply(M,a);
137     int m=SZ(M);
138     poly res(n);
139     fore(j,0,n) res[j]=(j+m-1<SZ(c))?c[j+m-1]:0;
140     return res;
141 }
142
143 void downsweep(const EvalTree &et, int v, poly q,
144     vector<tf> &out){
145     if(v>=et.k){ out[v-et.k]=q[0]; return; }
146     poly ql=mult(et.tree[2*v+1],q,SZ(et.tree[2*v+1])-1);
147     poly qr=mult(et.tree[2*v], q,SZ(et.tree[2*v+1])
148         -1);
149     poly().swap(q);
150     downsweep(et,2*v,mult(ql),out);
151     downsweep(et,2*v+1,mult(qr),out);
152 }
153
154 vector<tf> evaluate(const poly &a, const EvalTree &
155     et){
156     int k=et.k;
157     if(!k) return {};
158     poly f=SZ(a)>=SZ(et.tree[1])?divide(a,et.tree[1])
159         .snd:a;
160     poly rm=et.tree[1]; reverse(ALL(rm));
161     poly h=invert(rm,SZ(f));
162     poly().swap(rm);
163     reverse(ALL(h));
164     poly q1=mult(h,f,SZ(et.tree[1])-1);
165     poly().swap(h); poly().swap(f);
166     vector<tf> out(k);
167     downsweep(et,1,mult(q1),out);
168     return out;
169 }
170
171 // Multi-point polynomial evaluation for
172 // arbitrarily large points
173 vector<tf> evaluate(const poly &a, const vector<tf>
174     &x){
175     if(x.empty()) return {};
176     EvalTree et(x);
177     return evaluate(a,et);
178 }
179
180 poly upsweep(EvalTree &et, int v, const vector<tf>
181     &c){
182     if(v>=et.k) return {c[v-et.k]};
183     poly L=upsweep(et,2*v,c), R=upsweep(et,2*v+1,c);
184     poly p1=multiply(et.tree[2*v],R); poly().swap(R);
185     poly p2=multiply(et.tree[2*v+1],L); poly().swap(L);
186     poly().swap(et.tree[2*v]); poly().swap(et.tree[2*
187         v+1]);
188     poly res=add(p1,p2);
189     return res;
190 }

```

```

183 poly interpolate(const vector<tf> &x, const vector<
184     tf> &y){
185     int k=SZ(x);
186     if(!k) return {0};
187     EvalTree et(x);
188     vector<tf> d=evaluate(derivative(et.tree[1]),et);
189     vector<tf> pre(k+1); pre[0]=1;
190     fore(i,0,k) pre[i+1]=mulmod(pre[i],d[i]);
191     tf iv=inv(pre[k]);
192     vector<tf> c(k);
193     for(int i=k-1;i>=0;i--){
194         c[i]=mulmod(y[i],mulmod(iv,pre[i]));
195         iv=mulmod(iv,d[i]);
196     }
197     return upsweep(et,1,c);
198 }
199
200 // Given any number g, perform multi-point
201 // polynomial evaluation
202 // for all points of the form g^i, for each i in
203 // [0, k)
204 // in O((n+k) * log(n+k))
205 vector<int> chirpTransform(const poly &p, int g,
206     int k){
207     int inv_g=inv(g), n=SZ(p)-1, sz=min(k,MOD-1);
208     poly ap(n+1), bp(n+sz);
209
210     fore(i,0,n+sz){
211         int exp=1ll*i*(i-1)/2%(MOD-1);
212         if(i<=n) ap[n-i]=mulmod(p[i],fpow(inv_g, exp));
213         bp[i]=fpow(g, exp);
214     }
215
216     poly cp=multiply(ap, bp);
217     vector<int> ans(k);
218
219     fore(i,0,sz){
220         int exp=1ll*i*(i-1)/2%(MOD-1), val=0;
221         if(n+i<SZ(cp)) val=cp[n+i];
222         val=mulmod(val, fpow(inv_g,exp));
223         ans[i]=val;
224     }
225     fore(i,sz,k) ans[i]=ans[i-MOD+1];
226     return ans;
227 }
228
229 // Multi-point polynomial evaluation for all points
230 // in [0, MOD)
231 // MOD needs to be a prime number
232 vector<int> evaluate_all_points(const poly &p){
233     int g=MOD_PRIMITIVE_ROOT, gk=1;
234     assert(g!=-1);
235
236     vector<int> ch=chirpTransform(p,g,MOD-1), ans(MOD
237         , p[0]);
238
239     fore(i,0,MOD-1){
240         ans[gk]=ch[i];
241         gk=mulmod(gk,g);
242     }
243     return ans;
244 }

```

3.7 Fast Hadamard Transform

```

1 ll c1[MAXN+9],c2[MAXN+9];//MAXN must be power of 2!
2 void fht(ll* p, int n, bool inv){
3     for(int l=1;2*l<=n;l*=2)for(int i=0;i<n;i+=2*l)
4         fore(j,0,l){
5             ll u=p[i+j],v=p[i+l+j];
6             if(!inv)p[i+j]=u+v,p[i+l+j]=u-v; // XOR
7             else p[i+j]=(u+v)/2,p[i+l+j]=(u-v)/2;
8             //if(!inv)p[i+j]=v,p[i+l+j]=u+v; // AND
9             //else p[i+j]=-u+v,p[i+l+j]=u;
10            //if(!inv)p[i+j]=u+v,p[i+l+j]=u; // OR
11            //else p[i+j]=v,p[i+l+j]=u-v;
12        }
13 }
14 // like polynomial multiplication, but XORing
15 // instead of adding them (also ANDing, ORing)
16 vector<ll> multiply(vector<ll>& p1, vector<ll>& p2)
17 {
18     int n=1<<(32-__builtin_clz(max(SZ(p1),SZ(p2))-1))
19     ;
20     fore(i,0,n)c1[i]=0,c2[i]=0;
21     fore(i,0,SZ(p1))c1[i]=p1[i];
22     fore(i,0,SZ(p2))c2[i]=p2[i];
23     fht(c1,n,false);fht(c2,n,false);
24     fore(i,0,n)c1[i]*=c2[i];
25     fht(c1,n,true);
26     return vector<ll>(c1,c1+n);
27 }

```

3.8 Karatsuba

```

1 typedef ll tp;
2 #define add(n,s,d,k) fore(i,0,n)(d)[i]+=(s)[i]*k
3 tp* ini(int n){tp *r=new tp[n];fill(r,r+n,0);return
4     r;}
5 void karatsura(int n, tp* p, tp* q, tp* r){
6     if(n<=0)return;
7     if(n<35)fore(i,0,n)fore(j,0,n)r[i+j]+=p[i]*q[j];
8     else {
9         int nac=n/2,nbd=n-n/2; tp *a=p,*b=p+nac,*c=q,*d
10        =q+nac;
11        tp *ab=ini(nbd+1),*cd=ini(nbd+1),*ac=ini(nac*2)
12        ,*bd=ini(nbd*2);
13        add(nac,a,ab,1);add(nbd,b,ab,1); add(nac,c,cd
14        ,1);add(nbd,d,cd,1);
15        karatsura(nac,a,c,ac);karatsura(nbd,b,d,bd);
16        add(nac*2,ac,r+nac,-1);add(nbd*2,bd,r+nac,-1);
17        add(nac*2,ac,r,1);add(nbd*2,bd,r+nac*2,1);
18        karatsura(nbd+1,ab,cd,r+nac);
19        free(ab);free(cd);free(ac);free(bd);
20    }
21 }
22 vector<tp> multiply(vector<tp> p0, vector<tp> p1){
23     int n=max(SZ(p0),SZ(p1)); tp *p=ini(n),*q=ini(n)
24     ,*r=ini(2*n);
25     fore(i,0,p0.size())p[i]=p0[i];
26     fore(i,0,p1.size())q[i]=p1[i];
27     karatsura(n,p,q,r);
28     vector<tp> rr(r,r+p0.size()+p1.size()-1);
29     free(p);free(q);free(r);
30     return rr;
31 }

```

3.9 Diophantine

```

1 ii extendedEuclid (ll a, ll b){//a*x+b*y=gcd(a,b)
2     if(!b) return {1,0};
3     auto p=extendedEuclid(b,a%b); ll x=p.snd, y=p.fst
4     -(a/b)*x;
5     if(a*x+b*y<0) x=-x, y=-y;
6     return {x,y};
7 }
8 ii diophantine(ll a,ll b,ll r) {// =r inst of gcd
9     ll d=gcd(a,b); a/=d; b/=d; r/=d;
10    auto p = extendedEuclid(a,b); p.fst*=r; p.snd*=r;
11    return p; // solutions: p+t*(-b,a)/d
12 }

```

3.10 Chinese remainder theorem

```

1 #define mod(a,m) (((a)%m+m)%m)
2 #define inv(a,m) mod(diophantine(a,m,1).fst,m)
3 ii sol(ll a, ll x1, ll m){ //requires dioph
4     ll d=gcd(a,m);if(d<2)return{mod(x1*inv(a,m),m),m
5     };
6     return x1%d?ii({-1LL,-1LL}):sol(a/d,x1/d,m/d);
7 }
8 ii crt(vv as, vv bs, vv ms){ //returns:(sol,lcm)
9     ll x1=0,m1=1;
10    fore(i,0,SZ(ms)){
11        auto [x2,m2]=sol(as[i],bs[i],ms[i]);
12        if(m2<0|| (x1-x2)%gcd(m1,m2))return{-1,-1};
13        if(m1==m2)continue;; ll l=lcm(m1,m2);
14        ll k=diophantine(m2,-m1,x1-x2).snd;
15        x1=mod((x1)m1*k+x1,l);m1=l;
16    }
17    return {x1,m1};
18 }// as[i]*x=bs[i] (ms[i]); assumes lcm fits in ll

```

3.11 Mobius

```

1 short mu[MAXN] = {0,1};
2 void mobius(){
3     fore(i,1,MAXN)if(mu[i])for(int j=i+i;j<MAXN;j+=i)
4         mu[j]-=mu[i];
5 }

```

3.12 Simplex

```

1 vector<int> X,Y; vector<double> b,c; vector<vector<
2     double> > A;
3 double z; int n,m;
4 void pivot(int x,int y){
5     swap(X[y],Y[x]); b[x]/=A[x][y];
6     fore(i,0,m)if(i!=y)A[x][i]/=A[x][y];
7     A[x][y]=1/A[x][y];
8     fore(i,0,n)if(i!=x&&abs(A[i][y])>EPS){
9         b[i]-=A[i][y]*b[x];
10        fore(j,0,m)if(j!=y)A[i][j]-=A[i][y]*A[x][j];
11        A[i][y]-=A[i][y]*A[x][y];
12    }
13    z+=c[y]*b[x];
14    fore(i,0,m)if(i!=y)c[i]-=c[y]*A[x][i];
15    c[y]=-c[y]*A[x][y];
16 }
17 pair<double,vector<double> > simplex( // maximize c
18     ^T x s.t. Ax<=b, x>=0
19     vector<vector<double> > _A, vector<double> _b,
20     vector<double> _c){

```

```

18 // returns pair (max value, solution vector)
19 A=_A;b=_b;c=_c; n=b.size();m=c.size();z=0.;
20 X=vector<int>(m);Y=vector<int>(n);
21 fore(i,0,m)X[i]=i;
22 fore(i,0,n)Y[i]=i+m;
23 while(1){
24     int x=-1,y=-1; double mn=-EPS;
25     fore(i,0,n)if(b[i]<mn)mn=b[i],x=i;
26     if(x<0)break;
27     fore(i,0,m)if(A[x][i]<=EPS){y=i;break;}
28     assert(y>=0); // no solution to Ax<=b
29     pivot(x,y);
30 }
31 while(1){
32     double mx=EPS,mn=1e200; int x=-1,y=-1;
33     fore(i,0,m)if(c[i]>mx)mx=c[i],y=i;
34     if(y<0)break;
35     fore(i,0,n)if(A[i][y]>EPS&&b[i]/A[i][y]<mn)mn=b
        [i]/A[i][y],x=i;
36     assert(x>=0); // c^T x is unbounded
37     pivot(x,y);
38 }
39 vector<double> r(m);
40 fore(i,0,n)if(Y[i]<m)r[Y[i]]=b[i];
41 return {z,r};
42 }
    
```

3.13 Discrete log

```

1 //returns x such that a^x = b (mod m) or -1 if
  //inexistent
2 ll discrete_log(ll a,ll b,ll m) {
3     a%=m, b%=m;
4     if(b == 1) return 0;
5     int cnt=0; ll tmp=1;
6     for(int g=__gcd(a,m);g!=1;g=__gcd(a,m)) {
7         if(b%g) return -1;
8         m/=g, b/=g; tmp = tmp*a/g%m; ++cnt;
9         if(b == tmp) return cnt;
10    }
11    map<ll,int> w; int s=ceil(sqrt(m)); ll base=b;
12    fore(i,0,s) w[base] = i, base=base*a%m;
13    base=fastpow(a,s,m); ll key=tmp;
14    fore(i,1,s+2) {
15        key=base*key%m;
16        if(w.count(key)) return i*s-w[key]+cnt;
17    }
18    return -1;
19 }
    
```

3.14 Matrix reduce

```

1 double reduce(vector<vector<double> >& x){
2     int n=x.size(),m=x[0].size();
3     int i=0,j=0;double r=1.;
4     while(i<n&&j<m){
5         int l=i;
6         fore(k,i+1,n)if(abs(x[k][j])>abs(x[l][j]))l=k;
7         if(abs(x[l][j])<EPS){j++;r=0.;continue;}
8         if(l!=i){r=-r;swap(x[i],x[l]);}
9         r*=x[i][j];
10        fore(int k=m-1;k>=j;k--)x[i][k]/=x[i][j];
11        fore(k,0,n){
12            if(k==i)continue;
    
```

```

13        fore(int l=m-1;l>=j;l--)x[k][l]-=x[k][j]*x[i][
        l];
14    }
15    i++;j++;
16 }
17 return r; // returns determinant
18 }
    
```

3.15 Berlekamp Massey

```

1 typedef vector<int> vi;
2 vi BM(vi x){
3     vi ls,cur;int lf,ld;
4     fore(i,0,SZ(x)){
5         ll t=0;
6         fore(j,0,SZ(cur))t=(t+x[i-j-1]*(ll)cur[j])%MOD;
7         if((t-x[i])%MOD==0)continue;
8         if(!SZ(cur)){cur.resize(i+1);lf=i;ld=(t-x[i])%MOD
                ;continue;}
9         ll k=-(x[i]-t)*fast_pow(ld,MOD-2)%MOD; vi c(i-
                lf-1);c.pb(k);
10        fore(j,0,SZ(ls))c.pb((-ls[j]*k)%MOD);
11        if(SZ(c)<SZ(cur))c.resize(SZ(cur));
12        fore(j,0,SZ(cur))c[j]=(c[j]+cur[j])%MOD;
13        if(i-lf+SZ(ls)>=SZ(cur))ls=cur,lf=i,ld=(t-x[i])%
                MOD;
14        cur=c;
15    }
16    fore(i,0,SZ(cur))cur[i]=(cur[i]%MOD+MOD)%MOD;
17    return cur;
18 }
    
```

3.16 Linear Rec

```

1 //init 0(n^2log) query(n^2 logk)
2 //terms: first n term; trans: transition function;
  //MOD; LOG=mxlog of k
3 //output calc(k): kth term mod MOD
4 //ej: {1,1} {2,1} an=2*a_(n-1)+a_(n-2); calc(3)=3
  //calc(10007)=71480733
5 struct LinearRec{
6     typedef vector<int> vi;
7     int n; vi terms, trans; vector<vi> bin;
8     vi add(vi &a, vi &b){
9         vi res(n*2+1);
10        fore(i,0,n+1)fore(j,0,n+1)res[i+j]=(res[i+j]*1LL
                +(ll)a[i]*b[j])%MOD;
11        for(int i=2*n; i>n; --i){
12            fore(j,0,n)res[i-1-j]=(res[i-1-j]*1LL+(ll)res[i
                ]*trans[j])%MOD;
13            res[i]=0;
14        }
15        res.erase(res.begin()+n+1,res.end());
16        return res;
17    }
18    LinearRec(vi &terms, vi &trans):terms(terms),
        trans(trans){
19        n=SZ(trans);vi a(n+1);a[1]=1; bin.pb(a);
20        fore(i,1,LOG)bin.pb(add(bin[i-1],bin[i-1]));
21    }
22    int calc(int k){
23        vi a(n+1);a[0]=1;
24        fore(i,0,LOG)if((k>>i)&1)a=add(a,bin[i]);
25        int ret=0;
26        fore(i,0,n)ret=((ll)ret+(ll)a[i+1]*terms[i])%MOD;
    
```

```

27     return ret;
28 }
29 };

```

3.17 Tonelli Shanks

```

1  ll legendre(ll a, ll p){
2      if(a%p==0)return 0; if(p==2)return 1;
3      return fpow(a,(p-1)/2,p);
4  }
5  ll tonelli_shanks(ll n, ll p){ // sqrt(n) mod p (p
6      must be a prime)
7      assert(legendre(n,p)==1); if(p==2)return 1;
8      ll s=__builtin_ctzll(p-1), q=(p-1LL)>>s, z=rnd(1,
9          p-1);
10     if(s==1)return fpow(n,(p+1)/4LL,p);
11     while(legendre(z,p)!=p-1)z=rnd(1,p-1);
12     ll c=fpow(z,q,p),r=fpow(n,(q+1)/2,p),t=fpow(n,q,p
13         ),m=s;
14     while(t!=1){
15         ll i=1, ts=(t*t)%p;
16         while(ts!=1)i++,ts=(ts*ts)%p;
17         ll b=c;
18         fore(_,0,m-i-1)b=(b*b)%p;
19         r=r*b%p;c=b*b%p;t=t*c%p;m=i;
20     }
21     return r;
22 }

```

3.18 Prime count

```

1  // Count number of primes in [1,n]
2  // O(n^(2/3)) time
3  // O(sqrt(n)) memory
4  ll prime_count(ll n) {
5      if (n <= 1) return 0;
6      int v = sqrtl(n), s = (v+1)>>1, pc=0;
7      vector<int> smalls(s), roughs(s), skip(v+1);
8      vector<ll> larges(s);
9      fore(i,0,s){
10         smalls[i]=i;
11         roughs[i]=2*i+1;
12         larges[i]=(n/roughs[i]-1)>>1;
13     }
14     for(int p=3;p<=v;p+=2) if(!skip[p]){
15         int q=p*p;
16         if(1ll*q*q>n) break;
17         skip[p]=1;
18         for(int i=q;i<=v;i+=2*p) skip[i]=1;
19         int ns=0;
20         fore(k,0,s){
21             int i=roughs[k];
22             if(skip[i]) continue;
23             ll d=1ll*i*p;
24             larges[ns]=larges[k]-(d<=v?larges[
25                 smalls[d]>>1]-pc):smalls[(n/d-1)
26                 >>1]+pc;
27             roughs[ns++]=i;
28         }
29         s=ns;
30         for(int i=(v-1)>>1, j=((v/p)-1)|1; j>=p; j
31             -=2){
32             int c=smalls[j]>>1]-pc;
33             for(int e=(j*p)>>1;i>=e;i--) smalls[i
34                 ]-=c;
35         }
36     }
37 }

```

```

31     }
32     pc++;
33 }
34 larges[0]+=1ll*(s+2*(pc-1))*(s-1)>>1;
35 fore(k,1,s) larges[0]-=larges[k];
36 fore(l,1,s){
37     int q=roughs[l];
38     ll m=n/q, t=0;
39     int e=smalls[(m/q-1)>>1]-pc;
40     if(e<1+1)break;
41     fore(k,l+1,e+1) t+=smalls[(m/roughs[k]-1)
42         >>1];
43     larges[0]+=t-1ll*(e-1)*(pc+l-1);
44 }
45 return larges[0]+1;
46 }

```

3.19 Points Under Line

```

1  // # of lattice points s.t. ax+by<=c, x,y>0 (a,b is
2      positive integer)
3  ll f(ll a, ll b, ll c){
4      if(c<=0) return 0;
5      if(a<b) swap(a, b);
6      ll m=c/a, k=(a-1)/b, h=(c-a*m)/b;
7      if(a==b) return m*(m-1)/2;
8      return f(b,a-b*k,c-b*(k*m+h))+k*m*(m-1)/2+m*h;
9  }
10 // # of lattice points s.t. ax+by<=c, 0<x<=X, 0<y<=
11     Y (a,b integers > 0)
12 ll g(ll a, ll b, ll c, ll X, ll Y){
13     if(a*X+b*Y<=c) return X*Y;
14     return f(a,b,c)-f(a,b,c-a*X)-f(a,b,c-b*Y)+f(a,b,c
15         -a*X-b*Y);
16 }

```

3.20 Theorems and Formulas

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

$$\sum_{i=0}^k \binom{n+i}{i} = \binom{n+k+1}{k}$$

$$\left[\begin{matrix} n \\ k \end{matrix} \right] = \text{perm of } n \text{ elements with } k \text{ cycles}$$

$$\left[\begin{matrix} n+1 \\ k \end{matrix} \right] = n \left[\begin{matrix} n \\ k \end{matrix} \right] + \left[\begin{matrix} n \\ k-1 \end{matrix} \right]$$

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \text{partitions of an } n\text{-element set into } k \text{ parts}$$

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \frac{1}{k!} \sum_{i=0}^k (-1)^i \binom{k}{i} (k-i)^n.$$

$$\left\{ \begin{matrix} n+1 \\ k \end{matrix} \right\} = k \left\{ \begin{matrix} n \\ k \end{matrix} \right\} + \left\{ \begin{matrix} n \\ k-1 \end{matrix} \right\}$$
 Integers $d_1 \geq \dots \geq d_n \geq 0$ can be the degree sequence of a finite simple graph on n vertices $\iff$

$$d_1 + \dots + d_n \text{ is even and for every } k \text{ in } 1 \leq k \leq n$$

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k)$$

$$a^n = a^{\varphi(m)+n \bmod \varphi(m)} \pmod{m} \text{ if } n > \lg(m)$$
 Catalan numbers: $C_n = \frac{\binom{2n}{n}}{n+1} = \binom{2n}{n} - \binom{2n}{n+1}$
 Misere Nim: if $\exists a_i > 1$ then normal nim; else the condition is reversed.
 Derangements: Num of permutations of $n = 0, 1, 2, \dots$ elements without fixed points is
 1, 0, 1, 2, 9, 44, 265, 1854, 14833, Recurrence:

$D_n = (n-1)(D_{n-1} + D_{n-2}) = n * D_{n-1} + (-1)^n$. Collary:
 number of permutations with exactly k fixed points $nC_k D_{n-k}$
 Eulerian numbers: $E(n, k)$ is the number of permutations
 with exactly k descents ($i: \pi_i < \pi_{i+1}$), ascents ($\pi_i > \pi_{i+1}$) /
 excedances ($\pi > i$) / $k+1$ weak excedances ($\pi \geq i$).

$$E_{n,k} = (k+1)E_{n-1,k} + (n-k)E_{n-1,k-1}$$

$$\log(1-x^k) = -\sum_{j=1}^{\infty} \frac{x^{kj}}{j}$$

Bolitas en cajitas acotado, N bolitas, n cajitas, r es la cota
 por cajita.

$$\sum_{k=0}^{\lfloor N/(r+1) \rfloor} (-1)^k \binom{n}{k} \binom{N-k(r+1)+n-1}{n-1}$$

Catalan's triangle: number of rbs of n pairs with k pairs at
 depth 0.

$$cat(n, k) = \frac{k}{2n-k} \binom{2n-k}{n} = \binom{2n-k-1}{n-k} - \binom{2n-k-1}{n-k-1}$$

4 Geometry

4.1 Point

```

1 struct pt { // for 3D add z coordinate
2     double x,y;
3     pt(double x, double y):x(x),y(y){}
4     pt(){}
5     double norm2(){return *this**this;}
6     double norm(){return sqrt(norm2());}
7     bool operator==(pt p){return abs(x-p.x)<=EPS&&abs
        (y-p.y)<=EPS;}
8     pt operator+(pt p){return pt(x+p.x,y+p.y);}
9     pt operator-(pt p){return pt(x-p.x,y-p.y);}
10    pt operator*(double t){return pt(x*t,y*t);}
11    pt operator/(double t){return pt(x/t,y/t);}
12    double operator*(pt p){return x*p.x+y*p.y;}
13    // pt operator^(pt p){ // only for 3D
14    //     return pt(y*p.z-z*p.y,z*p.x-x*p.z,x*p.y-y*p.x
        );}
15    double angle(pt p){ // redefine acos for values
        out of range
16        return acos(*this*p/(norm()*p.norm()));}
17    pt unit(){return *this/norm();}
18    double operator%(pt p){return x*p.y-y*p.x;}
19    bool operator<(pt p)const{ // for convex hull
20        return x<p.x-EPS|| (abs(x-p.x)<=EPS&&y<p.y-EPS)
        ;}
21    bool left(pt p, pt q){return (q-p)%(*this-p)>EPS
        ;}
22    pt rot(pt r){return pt(*this*r,*this*r);}
23    pt rot(double a){return rot(pt(sin(a),cos(a)));}
24 };
25 pt ccw90(1,0), cw90(-1,0);
    
```

4.2 Line

```

1 int sgn2(double x){return x<0?-1:1;}
2 struct ln {
3     pt p,pq;
4     ln(pt p, pt q):p(p),pq(q-p){}
5     ln(){}
6     bool has(pt r){return dist(r)<=EPS;}
7     bool seghas(pt r){return has(r)&&(r-p)*(r-(p+pq))
        <=EPS;}
8    //bool operator/(ln l){return (pq.unit()^l.pq.unit
        ()).norm()<=EPS;}//3D
9    bool operator/(ln l){return abs(pq.unit()%l.pq.
        unit())<=EPS;} // 2D
10   bool operator==(ln l){return *this/l&&has(l.p);}
    
```

```

11   pt operator^(ln l){ // intersection
12       if(*this/l)return pt(DINF,DINF);
13       pt r=l.p+l.pq*((p-l.p)%pq/(l.pq%pq));
14       // if(!has(r)){return pt(NAN,NAN,NAN);} // check
        only for 3D
15       return r;
16   }
17   double angle(ln l){return pq.angle(l.pq);}
18   int side(pt r){return has(r)?0:sgn2(pq%(r-p));}
        // 2D
19   pt proj(pt r){return p+pq*((r-p)%pq/pq.norm2());}
20   pt ref(pt r){return proj(r)*2-r;}
21   double dist(pt r){return (r-proj(r)).norm();}
22   // double dist(ln l){ // only 3D
23   //     if(*this/l)return dist(l.p);
24   //     return abs((l.p-p)*(pq^l.pq))/(pq^l.pq).norm
        (});
25   // }
26   ln rot(auto a){return ln(p,p+pq.rot(a));} //2D
27 };
28 ln bisector(ln l, ln m){ // angle bisector
29     pt p=l.m; return ln(p,p+l.pq.unit()+m.pq.unit());
30 }
31 ln bisector(pt p, pt q){//segment bisector (2D)
32     return ln((p+q)*.5,p).rot(ccw90);
33 }
    
```

4.3 Circle

```

1 struct circle {
2     pt o;double r;
3     circle(pt o, double r):o(o),r(r){}
4     circle(pt x,pt y,pt z){o=bisector(x,y)^bisector(x
        ,z);r=(o-x).norm();}
5     bool has(pt p){return (o-p).norm()<=r+EPS;}
6     vector<pt> operator^(circle c){ // ccw
7         vector<pt> s; double d=(o-c.o).norm();
8         if(d>r+c.r+EPS||d+min(r,c.r)+EPS<max(r,c.r))
            return s;
9         double x=(d*d-c.r*c.r+r*r)/(2*d), y=sqrt(r*r-x*
            x);
10        pt v=(c.o-o)/d; s.pb(o+v*x-v.rot(ccw90)*y);
11        if(y>EPS)s.pb(o+v*x+v.rot(ccw90)*y);
12        return s;
13    }
14    vector<pt> operator^(ln l){
15        vector<pt> s; pt p=l.proj(o); double d=(p-o).
            norm();
16        if(d-EPS>r)return s;
17        if(abs(d-r)<=EPS){s.pb(p);return s;}
18        d=sqrt(r*r-d*d); s.pb(p+l.pq.unit()*d); s.pb(p-
            l.pq.unit()*d);
19        return s;
20    }
21    vector<pt> tang(pt p){
22        double d=sqrt((p-o).norm2()-r*r); return *this^
            circle(p,d);
23    }
24    bool in(circle c){ // non strict
25        double d=(o-c.o).norm(); return d+r<=c.r+EPS;
26    }
27    double intertriangle(pt a, pt b){ // area of
        intersection with oab
28        if(abs((o-a)%(o-b))<=EPS)return 0.;
    
```



```

29 vector<pt> q={a},w=*this~ln(a,b);
30 if(w.size()==2)for(auto p:w)if((a-p)*(b-p)<-EPS
    )q.pb(p);
31 q.pb(b);
32 if(q.size()==4&&(q[0]-q[1])*(q[2]-q[1])>EPS)
    swap(q[1],q[2]);
33 double s=0;
34 fore(i,0,q.size()-1){
35     if(!has(q[i])||!has(q[i+1]))s+=r*r*(q[i]-o).
        angle(q[i+1]-o)/2;
36     else s+=abs((q[i]-o)%(q[i+1]-o)/2);
37 }
38 return s;
39 }
40 };
41 vector<double> intercircles(vector<circle> c){
42     vector<double> r(SZ(c)+1); // r[k]: area covered
        by at least k circles
43     fore(i,0,SZ(c)){//O(n^2 log n) high constant!
44         int k=1;Cmp s(c[i].o);
45         vector<pair<pt,int> > p={
46             {c[i].o+pt(1,0)*c[i].r,0}, {c[i].o-pt(1,0)*c[
                i].r,0}};
47         fore(j,0,SZ(c))if(j!=i){
48             bool b0=c[i].in(c[j]),b1=c[j].in(c[i]);
49             if(b0&&(!b1||i<j))k++;
50             else if(!b0&&!b1){
51                 auto v=c[i]^c[j];
52                 if(SZ(v)==2){
53                     p.pb({v[0],1});p.pb({v[1],-1});
54                     if(s(v[1],v[0]))k++;
55                 }
56             }
57         }
58         sort(p.begin(),p.end(),
59             [&](pair<pt,int> a, pair<pt,int> b){return s(
                a.fst,b.fst)});
60         fore(j,0,SZ(p)){
61             pt p0=p[j?j-1:SZ(p)-1].fst,p1=p[j].fst;
62             double a=(p0-c[i].o).angle(p1-c[i].o);
63             r[k]+=(p0.x-p1.x)*(p0.y+p1.y)/2+c[i].r*c[i].r
                *(a-sin(a))/2;
64             k+=p[j].snd;
65         }
66     }
67     return r;
68 }

```

4.4 Polygon

```

1 int sgn(double x){return x<-EPS?-1:x>EPS;}
2 struct pol {
3     int n;vector<pt> p;
4     pol(){}
5     pol(vector<pt> _p){p=_p;n=p.size();}
6     pt centroid(){ // (barycenter)
7         pt r(0,0);double t=0;
8         fore(i,0,n){
9             r=r+(p[i]+p[(i+1)%n])*(p[i]%p[(i+1)%n]);
10            t+=p[i]%p[(i+1)%n];
11        }
12        return r/t/3;
13    }
14    bool has(pt q){ // O(n)

```

```

15         fore(i,0,n)if(ln(p[i],p[(i+1)%n]).seghas(q))
            return true;
16         int cnt=0;
17         fore(i,0,n){
18             int j=(i+1)%n, k=sgn((q-p[j])%(p[i]-p[j]));
19             int u=sgn(p[i].y-q.y),v=sgn(p[j].y-q.y);
20             if(k>0&&u<0&&v==0)cnt++;
21             if(k<0&&v<0&&u==0)cnt--;
22         }
23         return cnt!=0;
24     }
25     void normalize(){ // (call before haslog, remove
        collinear first)
26         if(p[2].left(p[0],p[1]))reverse(p.begin(),p.end
            ());
27         int pi=min_element(p.begin(),p.end())-p.begin()
            ;
28         vector<pt> s(n);
29         fore(i,0,n)s[i]=p[(pi+i)%n];
30         p.swap(s);
31     }
32     bool haslog(pt q){ // O(log(n)) only CONVEX. Call
        normalize first
33         if(q.left(p[0],p[1])||q.left(p.back(),p[0]))
            return false;
34         int a=1,b=p.size()-1;//returns true if point on
            boundary
35         while(b-a>1){//(change sign of EPS in left
            int c=(a+b)/2;//to return false in such case)
36             if(!q.left(p[0],p[c]))a=c;
37             else b=c;
38         }
39         return !q.left(p[a],p[a+1]);
40     }
41 }
42 pt farthest(pt v){ // O(log(n)) only CONVEX
43     if(n<10){
44         int k=0; fore(i,1,n)if(v*(p[i]-p[k])>EPS)k=i;
45         return p[k];
46     }
47     if(n==SZ(p))p.pb(p[0]);
48     pt a=p[1]-p[0]; int s=0,e=n,ua=v*a>EPS;
49     if(!ua&&v*(p[n-1]-p[0])<=EPS)return p[0];
50     while(1){
51         int m=(s+e)/2;pt c=p[m+1]-p[m], uc=v*c>EPS;
52         if(!uc&&v*(p[m-1]-p[m])<=EPS)return p[m];
53         if(ua&&(!uc||v*(p[s]-p[m])>EPS))e=m;
54         else if(ua||uc||v*(p[s]-p[m])>=-EPS)s=m,a=c,
            ua=uc;
55         else e=m;
56         assert(e>s+1);
57     }
58 }
59 pol cut(ln l){//cut CONVEX polygon by line l
60     vector<pt> q;//returns part at left of l.pq
61     fore(i,0,n){
62         int d0=sgn(l.pq(p[i]-l.p)),d1=sgn(l.pq(p[(i
            +1)%n]-l.p));
63         if(d0>0)q.pb(p[i]);
64         ln m(p[i],p[(i+1)%n]);
65         if(d0*d1<0&&!(1/m))q.pb(l^m);
66     }
67     return pol(q);
68 }
69 double intercircle(circle c){ // area of

```

```

        intersection with circle
70 double r=0.;
71 fore(i,0,n){
72     int j=(i+1)%n; double w=c.intertriangle(p[i],p
        [j]);
73     if((p[j]-c.o)%(p[i]-c.o)>0)r+=w;
74     else r-=w;
75 }
76 return abs(r);
77 }
78 double callipers(){ // square distance of most
        distant points
79 double r=0; // prereq: convex, ccw, NO
        COLLINEAR POINTS
80 for(int i=0,j=n<2?0:1;i<j;++i){
81     for(;;j=(j+1)%n){
82         r=max(r,(p[i]-p[j]).norm2());
83         if((p[(i+1)%n]-p[i])%(p[(j+1)%n]-p[j])<=EPS
            )break;
84     }
85 }
86 return r;
87 }
88 };
89 // Dynamic convex hull trick
90 vector<pt> w;
91 void add(pt q){ // add(q), O(log^2(n))
92     vector<pt> p={q};
93     while(!w.empty()&&SZ(w.back().p)<2*SZ(p)){
94         for(pt v:w.back().p)p.pb(v);
95         w.pop_back();
96     }
97     w.pb(pol(chull(p)));
98 }
99 ll query(pt v){ // max(q*v:q in w), O(log^2(n))
100     ll r=-INF;
101     for(auto& p:w)r=max(r,p.farthest(v)*v);
102     return r;
103 }

```

4.5 Plane

```

1 struct plane {
2     pt a,n; // n: normal unit vector
3     plane(pt a, pt b, pt c):a(a),n(((b-a)^(c-a)).unit
        ()){
4         plane(){}
5         bool has(pt p){return abs((p-a)*n)<=EPS;}
6         double angle(plane w){return acos(n*w.n);}
7         double dist(pt p){return abs((p-a)*n);}
8         pt proj(pt p){inter(ln(p,p+n),p);return p;}
9         bool inter(ln l, pt& r){
10             double x=n*(l.p+l.pq-a),y=n*(l.p-a);
11             if(abs(x-y)<=EPS)return false;
12             r=(l.p*x-(l.p+l.pq)*y)/(x-y); return true;
13         }
14         bool inter(plane w, ln& r){
15             pt nn=n^w.n;pt v=n*nn;double d=w.n*v;
16             if(abs(d)<=EPS)return 0;
17             pt p=a+v*(w.n*(w.a-a)/d);r=ln(p,p+nn);return 1;
18         }
19 };

```

4.6 Radial order of points

```

1 struct Cmp{//IMPORTANT: add const in pt operator -
2     pt r;
3     Cmp(pt r):r(r){}
4     int cuad(const pt &a)const {
5         if(a.x>0&&a.y>=0)return 0;
6         if(a.x<=0&&a.y>0)return 1;
7         if(a.x<0&&a.y<=0)return 2;
8         if(a.x>=0&&a.y<0)return 3;
9         assert(a.x==0&&a.y==0);
10        return -1;
11    }
12    bool cmp(const pt& p1, const pt& p2)const {
13        int c1=cuad(p1),c2=cuad(p2);
14        if(c1==c2)return p1.y*p2.x<p1.x*p2.y;
15        return c1<c2;
16    }
17    bool operator()(const pt& p1,const pt& p2)const {
18        return cmp(p1-r,p2-r);
19    }
20 };

```

4.7 Convex hull

```

1 // CCW order
2 // Includes collinear points (change sign of EPS in
        left to exclude)
3 vector<pt> chull(vector<pt> p){
4     vector<pt> r;
5     sort(p.begin(),p.end()); // first x, then y
6     p.erase(unique(ALL(p)), p.end());
7     fore(i,0,p.size()){ // lower hull
8         while(r.size()>=2&&r.back().left(r[r.size()-2],
            p[i]))r.pop_back();
9         r.pb(p[i]);
10    }
11    r.pop_back();
12    int k=r.size(),pr=k-1;
13    for(int i=p.size()-1;i>=0;--i){ // upper hull
14        while(r.size()>=k+2&&r.back().left(r[r.size()-2],
            p[i]))r.pop_back();
15        while(pr>=0&&p[i]<r[pr]) pr--;
16        if(pr<0||!(p[i]==r[pr])) r.pb(p[i]);
17    }
18    return r;
19 }

```

4.8 Convex hull reduced

```

1 vector<pt> chull(vector<pt> p){// CCW order
2     if(SZ(p)<3) return p;//incl collinear
3     vector<pt> r; sort(ALL(p)); int k=0,pr=-1;
4     fore(_,0,2){//first lower hull, then upper hull
5         fore(i,0,SZ(p)){
6             while(SZ(r)>1+k&&r.back().left(r[SZ(r)-2],p[i]
            ))r.pop_back();
7             while(pr>=0&&p[i]<r[pr]) pr--;
8             if(pr<0||!(p[i]==r[pr])) r.pb(p[i]);
9         }
10        if(!_r.pop_back(),k=SZ(r),pr=k-1,reverse(
            ALL(p));
11    }
12    return r;
13 }

```

4.9 MEC

```

1 pt bary(pt A, pt B, pt C, ld a, ld b, ld c) {
2     return (A*a + B*b + C*c) / (a + b + c);
3 }
4 pt circumcenter(pt A, pt B, pt C) { // intersection
5     of perpendicular bisectors
6     ld a=(B-C).norm2(), b=(C-A).norm2(), c=(A-B).
7     norm2();
8     return bary(A, B, C, a*(b+c-a), b*(c+a-b), c*(a
9     +b-c));
10 }
11 random_device rd;
12 mt19937 rng(rd());
13 pair<pt,ld> mec(vector<pt> ps) {
14     shuffle(ALL(ps),rng);
15     pt o=ps[0];
16     ld r=0, EPS=1+1e-8;
17     fore(i,0,SZ(ps))if((o-ps[i]).norm())>r*EPS) {
18         o=ps[i],r=0;
19         fore(j,0,i)if((o-ps[j]).norm())>r*EPS) {
20             o=(ps[i]+ps[j])/2;
21             r=(o-ps[i]).norm();
22             fore(k,0,j)if((o-ps[k]).norm())>r*EPS) {
23                 o=circumcenter(ps[i],ps[j],ps[k]);
24                 r=(o-ps[i]).norm();
25             }
26         }
27     }
28     return {o,r};
29 }
30 --this points not affect mec
31 centroid:bary(A,B,C,1,1,1);
32 incenter:bary(A,B,C,(B-C).norm(),(A-C).norm(),(A-B).
33     norm)
34 a=(B-C).norm2,b=(C-A).norm2,c=(A-B).norm2
35 orthocenter:bary(A,B,C,(a+b-c)*(c+a-b),(b+c-a)*(a+b
36     -c),(c+a-b)*(b+c-a))
37 a=(B-C).norm,b=(A-C).norm,c=(A-B).norm
38 excenter:bar(A,B,C,-a,b,c)
    
```

4.10 Dual from planar graph

```

1 vector<int> g[MAXN];int n; // input graph (must be
2     connected)
3 vector<int> gd[MAXN];int nd; // output graph
4 vector<int> nodes[MAXN]; // nodes delimiting region
5     (in CW order)
6 map<pair<int,int>,int> ps,es;
7 void get_dual(vector<pt> p){ // p: points
8     corresponding to nodes
9     ps.clear();es.clear();
10    fore(x,0,n){
11        Cmp pc(p[x]); // (radial order of points)
12        auto comp=[&](int a, int b){return pc(p[a],p[b
13            ]);};
14        sort(g[x].begin(),g[x].end(),comp);
15        fore(i,0,g[x].size())ps[{x,g[x][i]}]=i;
16    }
17    nd=0;
18    fore(xx,0,n)for(auto yy:g[xx])if(!es.count({xx,yy
19        })){
20        int x=xx,y=yy;gd[nd].clear();nodes[nd].clear();
21        while(!es.count({x,y})){
22            es[{x,y}]=nd;nodes[nd].pb(y);
23            int z=g[y][ (ps[{y,x}]+1)%g[y].size() ];x=y;y=z
    
```

```

19    }
20    nd++;
21 }
22 for(auto p:es){
23     pair<int,int> q={p.fst.snd,p.fst.fst}; assert(
24     es.count(q));
25     if(es[q]!=p.snd)gd[p.snd].pb(es[q]);
26 }
27 fore(i,0,nd){
28     sort(gd[i].begin(),gd[i].end());
29     gd[i].erase(unique(gd[i].begin(),gd[i].end()),
30     gd[i].end());
31 }
32 }
    
```

4.11 Halfplane intersection

```

1 // polygon intersecting left side of halfplanes
2 struct halfplane:public ln{
3     double angle;
4     halfplane(){}
5     halfplane(pt a,pt b){p=a; pq=b-a; angle=atan2(pq.
6     y,pq.x);}
7     bool operator<(halfplane b)const{return angle<b.
8     angle;}
9     bool out(pt q){return pq*(q-p)<-EPS;}
10 };
11 vector<pt> intersect(vector<halfplane> b){
12     vector<pt>bx={{DINF,DINF},{-DINF,DINF},{-DINF,-
13     DINF},{DINF,-DINF}};
14     fore(i,0,4) b.pb(halfplane(bx[i],bx[(i+1)%4]));
15     sort(ALL(b));
16     int n=SZ(b),q=1,h=0; vector<halfplane> c(SZ(b)
17     +10);
18     fore(i,0,n){
19         while(q<h&&b[i].out(c[h]^c[h-1])) h--;
20         while(q<h&&b[i].out(c[q]^c[q+1])) q++;
21         c[++h]=b[i];
22         if(q<h&&abs(c[h].pq*c[h-1].pq)<EPS){
23             if(c[h].pq*c[h-1].pq<0) return {};
24             h--;
25             if(b[i].out(c[h].p)) c[h]=b[i];
26         }
27     }
28     while(q<h-1&&c[q].out(c[h]^c[h-1]))h--;
29     while(q<h-1&&c[h].out(c[q]^c[q+1]))q++;
30     if(h-q<=1)return {};
31     c[h+1]=c[q]; vector<pt> s;
32     fore(i,q,h+1) s.pb(c[i]^c[i+1]);
33     return s;
34 }
    
```

4.12 KD Tree

```

1 // given a set of points, answer queries of nearest
2     point in O(log(n))
3 bool onx(pt a, pt b){return a.x<b.x;}
4 bool ony(pt a, pt b){return a.y<b.y;}
5 struct Node {
6     pt pp; ll x0=INF, x1=-INF, y0=INF, y1=-INF;
7     Node *first=0, *second=0;
8     ll distance(pt p){
9         ll x=min(max(x0,p.x),x1), y=min(max(y0,p.y),y1)
10         ;
    
```

```

9     return (pt(x,y)-p).norm2();
10 }
11 Node(vector<pt>&& vp):pp(vp[0]){
12     for(pt p:vp){
13         x0=min(x0,p.x); x1=max(x1,p.x); y0=min(y0,p.y);
14         y1=max(y1,p.y);
15     }
16     if(SZ(vp)>1){
17         sort(ALL(vp),x1-x0>y1-y0?onx:ony);
18         int m=SZ(vp)/2; first=new Node({vp.begin(),vp
19             .begin()+m});
20         second=new Node({vp.begin()+m,vp.end()});
21     }
22 };
23 struct KDTree {
24     Node* root;
25     KDTree(const vector<pt>& vp):root(new Node({ALL(
26         vp}))) {}
27     pair<ll,pt> search(pt p, Node *node){
28         if(!node->first){
29             //if(p==node->pp) {INF,pt()}; //avoid query
30             //point as answer
31             return {(p-node->pp).norm2(),node->pp};
32         }
33         Node *f=node->first, *s=node->second;
34         ll bf=f->distance(p), bs=s->distance(p);
35         if(bf>bs)swap(bf,bs),swap(f,s);
36         auto best=search(p,f);
37         if(bs<best.fst) best=min(best,search(p,s));
38         return best;
39     }
40     pair<ll,pt> nearest(pt p){return search(p,root);}
41 };

```

4.13 Closest points

```

1 // sort by x
2 ll closest(vector<ii> &p) {
3     int n = SZ(p);
4     set<ii> s;
5     ll best = 1e18;
6     int j = 0;
7     for(i, 0, n) {
8         ll d = ceil(sqrt(best));
9         while(p[i].fst - p[j].fst >= best) s.erase({p[j].
10             snd, p[j].fst}), j++;
11         auto it1=s.lower_bound({p[i].snd-d,p[i].fst});
12         auto it2=s.upper_bound({p[i].snd+d,p[i].fst});
13         for(auto it = it1; it != it2; ++it) {
14             ll dx = p[i].fst - it->snd;
15             ll dy = p[i].snd - it->fst;
16             best = min(best, dx * dx + dy * dy);
17         }
18         s.insert({p[i].snd, p[i].fst});
19     }
20     return best;
21 }

```

4.14 Minkowski Sum

```

1 // Compute Minkowski sum of two CONVEX polygons (
2 // remove collinear first)
3 // Returns answer in CCW order
4 void reorder(vector<pt> &p){

```

```

4     if(!p[2].left(p[0],p[1])) reverse(ALL(p));
5     int pos=0;
6     for(i,1,SZ(p)) if(pt(p[i].y,p[i].x) < pt(p[pos].
7         y,p[pos].x)) pos=i;
8     rotate(p.begin(), p.begin()+pos, p.end());
9 }
10 vector<pt> minkowski_sum(vector<pt> p, vector<pt> q
11     ){
12     if(min(SZ(p),SZ(q))<3){
13         vector<pt> v;
14         for(pt pp:p) for(pt qq:q) v.pb(pp+qq);
15         return hull(v);
16     }
17     reorder(p); reorder(q);
18     for(i,0,2) p.pb(p[i]), q.pb(q[i]);
19     vector<pt> r;
20     int i=0,j=0;
21     while(i+2<SZ(p)||j+2<SZ(q)){
22         r.pb(p[i]+q[j]);
23         auto cross=(p[i+1]-p[i])%(q[j+1]-q[j]);
24         i+=cross>=-EPS;
25         j+=cross<=EPS;
26     }
27     return r;
28 }

```

4.15 Theorems and Formulas

The n-dimensional volume of a ball of radius r is

$$V_n(r) = \frac{\pi^{n/2}}{\Gamma(\frac{n}{2} + 1)} r^n$$

where $\Gamma(n) = (n-1)!$ and $\Gamma(n + \frac{1}{2}) = (n - \frac{1}{2}) \cdot \dots \cdot \frac{1}{2} \cdot \pi^{\frac{1}{2}}$

If $\mathbf{v} \in \mathbb{R}^3$ and $\mathbf{k}$ is a unit vector describing an axis of rotation about which $\mathbf{v}$ rotates by an angle θ

according to the right hand rule:

$$\mathbf{v}_{\text{rot}} = \mathbf{v} \cos \theta + (\mathbf{k} \times \mathbf{v}) \sin \theta + \mathbf{k} (\mathbf{k} \cdot \mathbf{v})(1 - \cos \theta)$$

Spherical cone with sphere radius r,

height h and radius of circle a:

$$A = \pi r(2h + a) \text{ (includes lateral area)}$$

$$V = \frac{2}{3} \pi r^2 h$$

Surface/Solid of revolution

$$A = 2\pi Ld, L = \text{length of curve}$$

$$V = 2\pi Ad, A = \text{area of surface}$$

$$d = \text{distance of centroid to axis}$$

5 Strings

5.1 KMP

```

1 vector<int> kmppre(string& t){ // r[i]: longest
2     border of t[0,i]
3     vector<int> r(t.size()+1); r[0]=-1; int j=-1;
4     for(i,0,t.size()){
5         while(j>=0&&t[i]!=t[j])j=r[j];
6         r[i+1]=++j;
7     }
8     return r;
9 }
10 void kmp(string& s, string& t){ // find t in s
11     int j=0;vector<int> b=kmppre(t);
12     for(i,0,SZ(s)){
13         while(j>=0&&s[i]!=t[j])j=b[j];

```

```

13     if(++j==SZ(t))printf("Match at %d\n",i-j+1),j=b
        [j];
14 }
15 }
16 //KMP automaton
17 const int MAXN=1e5,MAXC=26;
18 int wh[MAXN+2][MAXC]; //wh[i][j] = jump when adding
    j to prefix of size i
19 void build(string &s){
20     fore(j,0,MAXC) wh[0][j]=0;
21     int lps=0; wh[0][s[0]-'a'] = 1;
22     fore(i,1,SZ(s)+1){
23         fore(j,0,MAXC) wh[i][j]=wh[lps][j];
24         if(i<SZ(s)) wh[i][s[i]-'a'] = i+1, lps = wh[lps]
            [s[i]-'a'];
25     }
26 }

```

5.2 Z function

```

1 vector<int> z_function(string& s){
2     int l=0,r=0,n=s.size();
3     vector<int> z(s.size(),0); // z[i] = max k: s[0,k
        ) == s[i,i+k)
4     fore(i,1,n){
5         if(i<=r)z[i]=min(r-i+1,z[i-l]);
6         while(i+z[i]<n&&s[z[i]]==s[i+z[i]])z[i]++;
7         if(i+z[i]-1>r)l=i,r=i+z[i]-1;
8     }
9     return z;
10 }

```

5.3 Hashing

```

1 #define bint __int128
2 struct Hash {
3     bint MOD=212345678987654321LL,P=17777771,PI
        =106955741089659571LL;
4     vector<bint> h,pi;
5     Hash(string& s){
6         assert((P*PI)%MOD==1);
7         h.resize(s.size()+1);pi.resize(s.size()+1);
8         h[0]=0;pi[0]=1; bint p=1;
9         fore(i,1,s.size()+1){
10             h[i]=(h[i-1]+p*s[i-1])%MOD;
11             pi[i]=(pi[i-1]*PI)%MOD; p=(p*P)%MOD;
12         }
13     }
14     ll get(int s, int e){
15         return ((h[e]-h[s]+MOD)%MOD)*pi[s]%MOD;
16     }
17 };

```

5.4 Manacher

```

1 //Max palindrome centered on i, p=pal-parity
2 vv manacher(string &s, bool p){
3     int n=SZ(s), l=-1,r=0;
4     vv d(n); // substr(l,r) last max pal
5     fore(i,0,n){
6         int k=(i<r?min(d[l+r-i+!p],r-i):p);
7         while(i+k<n&&k+i+p&&s[i-k-!p]==s[i+k]) k++;
8         d[i]=k;
9         if(i+k>r) r=i+k,l=i-k-!p;
10     }

```

```

11     return d;
12 }

```

5.5 Aho-Corasick

```

1 struct vertex {
2     map<char,int> next,go; int p,link; char pch;
3     vector<int> leaf;
4     vertex(int p=-1, char pch=-1):p(p),pch(pch),link
        (-1){}
5 };
6 vector<vertex> t;
7 void aho_init(){ //do not forget!!
8     t.clear();t.pb(vertex());
9 }
10 void add_string(string s, int id){
11     int v=0;
12     for(char c:s){
13         if(!t[v].next.count(c)) t[v].next[c]=t.size(),
14             t.pb(vertex(v,c));
15         v=t[v].next[c];
16     }
17     t[v].leaf.pb(id);
18 }
19 int go(int v, char c){
20     if(t[v].link<0)
21         if(!v||!t[v].p)t[v].link=0;
22         else t[v].link=go(get_link(t[v].p),t[v].pch);
23     return t[v].link;
24 }
25 int go(int v, char c){
26     if(!t[v].go.count(c))
27         if(t[v].next.count(c))t[v].go[c]=t[v].next[c];
28         else t[v].go[c]=v==0?0:go(get_link(v),c);
29     return t[v].go[c];
30 }

```

5.6 Suffix automaton

```

1 struct state {int len,link;map<char,int> next;}; //
    clear next!!
2 state st[2*MAXN]; int sz,last;
3 void sa_init(){ last=st[0].len=0;sz=1; st[0].link
    =-1; }
4 void sa_extend(char c){
5     int k=sz++,p; st[k].len=st[last].len+1;
6     for(p=last;p!=-1&&!st[p].next.count(c);p=st[p].
        link)st[p].next[c]=k;
7     if(p==-1)st[k].link=0;
8     else {
9         int q=st[p].next[c];
10        if(st[p].len+1==st[q].len)st[k].link=q;
11        else {
12            int w=sz++; st[w].len=st[p].len+1;
13            st[w].next=st[q].next;st[w].link=st[q].link;
14            for(p=-1&&st[p].next[c]==q;p=st[p].link)st[
                p].next[c]=w;
15            st[q].link=st[k].link=w;
16        }
17    }
18    last=k;
19 }

```

5.7 Palindromic Tree


```

1 struct palindromic_tree{
2     static const int SIGMA=26;
3     struct Node{
4         int len, link, to[SIGMA]; ll cnt;
5         Node(int len, int link=0, ll cnt=1):len(len),
6             link(link),cnt(cnt){
7             memset(to,0,sizeof(to));
8         }
9     };
10    vector<Node> ns; int last;
11    palindromic_tree():last(0){ns.pb(Node(-1));ns.pb(
12        Node(0));}
13    void add(int i, string &s){
14        int p=last, c=s[i]-'a';
15        while(s[i-ns[p].len-1]!=s[i])p=ns[p].link;
16        if(ns[p].to[c])last=ns[p].to[c],ns[last].cnt++;
17        else{
18            int q=ns[p].link;
19            while(s[i-ns[q].len-1]!=s[i])q=ns[q].link;
20            q=max(1,ns[q].to[c]);last=ns[p].to[c]=SZ(ns);
21            ns.pb(Node(ns[p].len+2,q,1));
22        }
23    };

```

5.8 Suffix array

```

1 #define RB(x) (x<n?r[x]:0)
2 void csort(vector<int>& sa, vector<int>& r, int k){
3     int n=sa.size(),top=max(255,n+1);
4     vector<int> f(top,0),t(n);
5     fore(i,0,n)f[RB(i+k)]++;
6     int sum=0;
7     fore(i,0,top)f[i]=(sum+=f[i])-f[i];
8     fore(i,0,n)t[f[RB(sa[i]+k)]]+=sa[i];
9     sa=t;
10 }
11 vector<int> constructSA(string& s){ // O(n logn)
12     int n=s.size(),rank;
13     vector<int> sa(n),r(n),t(n);
14     fore(i,0,n)sa[i]=i,r[i]=s[i];
15     for(int k=1;k<n;k*=2){
16         csort(sa,r,k);csort(sa,r,0);
17         t[sa[0]]=rank=1;
18         fore(i,1,n){
19             if(r[sa[i]]!=r[sa[i-1]]||RB(sa[i]+k)!=RB(sa[i-1]+k))rank++;
20             t[sa[i]]=rank;
21         }
22         r=t;
23         if(r[sa[n-1]]==n)break;
24     }
25     return sa;
26 }

```

5.9 LCP (Longest Common Prefix)

```

1 vector<int> computeLCP(string& s, vector<int>& sa){
2     int n=s.size(),L=0; vector<int> lcp(n),plcp(n),
3         phi(n);
4     phi[sa[0]]=-1;
5     fore(i,1,n)phi[sa[i]]=sa[i-1];
6     fore(i,0,n){
7         if(phi[i]<0){plcp[i]=0;continue;}
8         while(s[i+L]==s[phi[i]+L])L++;
9     }

```

```

8     plcp[i]=L; L=max(L-1,0);
9 }
10 fore(i,0,n)lcp[i]=plcp[sa[i]];
11 return lcp; // lcp[i]=LCP(sa[i-1],sa[i])
12 }

```

5.10 Minimum Lexicographic rotation

```

1 vector<ll> getminlex(vector<ll> s){
2     int n=SZ(s),k=0; fore(i,0,n) s.pb(s[i]);
3     vector<int> f(2*n,-1);
4     fore(j,1,2*n){
5         int i=f[j-k-1];
6         while(i>=0&&s[j]!=s[k+i+1]){
7             if(s[j]<s[k+i+1]) k=j-i-1;
8             i=f[i];
9         }
10        if(s[j]!=s[k+i+1]){
11            if(s[j]<s[k])k=j;
12            f[j-k]=-1;
13        } else f[j-k]=i+1;
14    }
15    vector<ll> ans; fore(i,0,n) ans.pb(s[k+i]);
16    return ans;
17 }

```

6 Flow

6.1 Matching $O(n^2)$

```

1 vector<int> g[MAXN]; // [0,n)->[0,m)
2 int n,m,mat[MAXN];bool vis[MAXN];
3 int match(int x){
4     if(vis[x])return 0;
5     vis[x]=true;
6     for(int y:g[x])if(mat[y]<0||match(mat[y])){mat[y]
7         =x;return 1;}
8     return 0;
9 }
10 vector<pair<int,int> > max_matching(){
11     vector<pair<int,int> > r; memset(mat,-1,sizeof(
12         mat));
13     fore(i,0,n)memset(vis,false,sizeof(vis)),match(i);
14     return r;

```

6.2 Matching (Hopcroft-Karp)

```

1 vector<int> g[MAXN]; // [0,n)->[0,m)
2 int n,m;
3 int mt[MAXN],mt2[MAXN],ds[MAXN];
4 bool bfs(){
5     queue<int> q; memset(ds,-1,sizeof(ds)); bool r=
6         false;
7     fore(i,0,n)if(mt2[i]<0)ds[i]=0,q.push(i);
8     while(!q.empty()){
9         int x=q.front();q.pop();
10        for(int y:g[x]){
11            if(mt[y]>=0&&ds[mt[y]]<0)ds[mt[y]]=ds[x]+1,q.
12                push(mt[y]);
13            else if(mt[y]<0)r=true;
14        }
15    }
16    return r;

```

```

15 }
16 bool dfs(int x){
17     for(int y:g[x])if(mt[y]<0||ds[mt[y]]==ds[x]+1&&
18         dfs(mt[y])){
19         mt[y]=x;mt2[x]=y; return true;
20     }
21     ds[x]=1<<30; return false;
22 }
23 int mm(){
24     int r=0; memset(mt,-1,sizeof(mt));memset(mt2,-1,
25         sizeof(mt2));
26     while(bfs()) fore(i,0,n)if(mt2[i]<0)r+=dfs(i);
27     return r;
28 }

```

6.3 Hungarian

```

1 typedef long double td; typedef vector<int> vi;
2 typedef vector<td> vd;
3 const td INF=1e100;//for maximum set INF to 0, and
4     negate costs
5 bool zero(td x){return fabs(x)<1e-9;}//change to x
6     ==0, for ints/ll
7 struct Hungarian{
8     int n; vector<vd> cs; vi L, R;
9     Hungarian(int N, int M):n(max(N,M)),cs(n,vd(n)),L
10         (n),R(n){
11         fore(x,0,N)fore(y,0,M)cs[x][y]=INF;
12     }
13 void set(int x,int y,td c){cs[x][y]=c;}
14 td assign() {
15     int mat=0; vd ds(n),u(n),v(n); vi dad(n),sn(n);
16     fore(i,0,n)u[i]=*min_element(ALL(cs[i]));
17     fore(j,0,n){
18         v[j]=cs[0][j]-u[0];fore(i,1,n)v[j]=min(v[j],
19             cs[i][j]-u[i]);
20     }
21     L=R=vi(n, -1);
22     fore(i,0,n)fore(j,0,n)
23         if(R[j]==-1&&zero(cs[i][j]-u[i]-v[j])){L[i]=j
24             ;R[j]=i;mat++;break;}
25     for(;mat<n;mat++){
26         int s=0, j=0, i;
27         while(L[s] != -1)s++;
28         fill(ALL(dad),-1);fill(ALL(sn),0);
29         fore(k,0,n)ds[k]=cs[s][k]-u[s]-v[k];
30         for(;;){
31             j = -1;
32             fore(k,0,n)if(!sn[k]&&(j==-1||ds[k]<ds[j]))
33                 j=k;
34             sn[j] = 1; i = R[j];
35             if(i == -1) break;
36             fore(k,0,n)if(!sn[k]){
37                 auto new_ds=ds[j]+cs[i][k]-u[i]-v[k];
38                 if(ds[k]>new_ds){ds[k]=new_ds;dad[k]=j;}
39             }
40         }
41         fore(k,0,n)if(k!=j&&sn[k]){auto w=ds[k]-ds[j]
42             ;v[k]+=w,u[R[k]]-=w;}
43         u[s] += ds[j];
44         while(dad[j]>=0){int d = dad[j];R[j]=R[d];L[R
45             [j]]=j;j=d;}
46         R[j]=s;L[s]=j;
47     }
48 }

```

```

39     td value=0;fore(i,0,n)value+=cs[i][L[i]];
40     return value;
41 }
42 };

```

6.4 Dinic

```

1 // Min cut: nodes with dist>=0 vs nodes with dist<0
2 // Matching MVC: left nodes with dist<0 + right
3     nodes with dist>0
4 struct Dinic{
5     struct edge {int to,rev;ll f,cap;}
6     vector<vector<edge>> g;
7     int src,dst; vector<int> dist,q,work;
8     Dinic(int x):g(x),dist(x),q(x),work(x){}
9     void add_edge(int s, int t, ll cap){
10         g[s].pb((edge){t,SZ(g[t]),0,cap});
11         g[t].pb((edge){s,SZ(g[s])-1,0,0});
12     }
13 bool dinic_bfs(){
14     fill(ALL(dist),-1);dist[src]=0; int qt=0;q[qt
15         ++]=src;
16     for(int qh=0;qh<qt;qh++){
17         int u=q[qh];
18         fore(i,0,SZ(g[u])){
19             edge &e=g[u][i];int v=g[u][i].to;
20             if(dist[v]<0&&e.f<e.cap)dist[v]=dist[u]+1,q
21                 [qt++]=v;
22         }
23     }
24     return dist[dst]>=0;
25 }
26 ll dinic_dfs(int u, ll f){
27     if(u==dst)return f;
28     for(int &i=work[u];i<SZ(g[u]);i++){
29         edge &e=g[u][i]; int v=e.to;
30         if(e.cap<=e.f)continue;
31         if(dist[v]==dist[u]+1){
32             ll df=dinic_dfs(v,min(f,e.cap-e.f));
33             if(df>0){e.f+=df;g[v][e.rev].f-=df;return
34                 df;}
35         }
36     }
37     return 0;
38 }
39 ll max_flow(int _src, int _dst){
40     src=_src;dst=_dst; ll result=0;
41     while(dinic_bfs()){
42         fill(ALL(work),0);
43         while(ll delta=dinic_dfs(src,INF))result+=
44             delta;
45     }
46     return result;
47 }
48 };

```

6.5 Min cost max flow

```

1 typedef ll tf; typedef ll tc;
2 const tf INFFLOW=1e9; const tc INFCOST=1e9;
3 struct MCF{
4     int n; vector<tc> prio, pot;
5     vector<tf> curflow;vector<int> prevege,prevnode;
6     priority_queue<pair<tc, int>, vector<pair<tc, int>
7         >>, greater<pair<tc, int>>> q;

```

```

7 struct edge{int to, rev; tf f, cap; tc cost;};
8 vector<vector<edge>> g;
9 MCF(int n):n(n),prio(n),curflow(n),prevedge(n),
    prevnode(n),pot(n),g(n){}
10 void add_edge(int s, int t, tf cap, tc cost) {
11     g[s].pb((edge){t,SZ(g[t]),0,cap,cost});
12     g[t].pb((edge){s,SZ(g[s])-1,0,0,-cost});
13 }
14 pair<tf,tc> get_flow(int s, int t) {
15     tf flow=0; tc flowcost=0;
16     while(1){
17         q.push({0, s}); fill(ALL(prio),INFCOST);
18         prio[s]=0; curflow[s]=INFFLOW;
19         while(!q.empty()) {
20             auto cur=q.top(); tc d=cur.fst; int u=cur.
                snd; q.pop();
21             if(d!=prio[u]) continue;
22             for(int i=0; i<SZ(g[u]); ++i) {
23                 edge &e=g[u][i]; int v=e.to;
24                 if(e.cap<=e.f) continue;
25                 tc nprio=prio[u]+e.cost+pot[u]-pot[v];
26                 if(prio[v]>nprio) {
27                     prio[v]=nprio; q.push({nprio, v});
28                     prevnode[v]=u; prevedge[v]=i;
29                     curflow[v]=min(curflow[u],e.cap-e.f);
30                 }
31             }
32         }
33         if(prio[t]==INFCOST) break;
34         fore(i,0,n) pot[i]+=prio[i];
35         tf df=min(curflow[t],INFFLOW-flow); flow+=df;
36         for(int v=t; v!=s; v=prevnode[v]) {
37             edge &e=g[prevnode[v]][prevedge[v]];
38             e.f+=df; g[v][e.rev].f-=df; flowcost+=df*e.
                cost;
39         }
40     }
41     return {flow,flowcost};
42 }
43 };

```

6.6 Min cost max flow cycles

```

1 typedef int tf;
2 typedef int tc;
3 const tc INFCOST = numeric_limits<tc>::max()/2;
4 const int scale=2;
5 struct mcSFlow{
6     int n, s, t; tc eps;
7     vector<int> isq, cur, co; vector<tf> ex; vector<
        tc> h; vector<vector<int>> hs;
8     struct edge{int to, rev; tf f; tc c;};
9     vector<vector<edge>> g;
10    mcSFlow(int n,int s,int t):eps(0),n(n),s(s),t(t),
        g(n){}
11    void add_edge(int a, int b, tc cost, tf cap){
12        if(a==b){assert(cost>=0); return;}
13        cost*=n; eps = max(eps, abs(cost));
14        g[a].pb({b, SZ(g[b]), cap, cost});
15        g[b].pb({a, SZ(g[a])-1, 0, -cost});
16    }
17    void add_flow(edge& e, tf f) {
18        edge &back = g[e.to][e.rev];
19        if (!ex[e.to] && f) hs[h[e.to]].push_back(e.to);

```

```

20    e.f -= f; ex[e.to] += f;
21    back.f += f; ex[back.to] -= f;
22    }
23    tf max_flow() {
24        ex.assign(n,0),h.assign(n,0),co.assign(2*n,0),cur
            .assign(n,0),hs.resize(2*n);
25        h[s] = n, ex[t] = 1, co[0] = n - 1;
26        for(auto &e:g[s]) add_flow(e, e.f);
27        if(hs[0].size()) for (int hi=0;hi>=0;) {
28            int u = hs[hi].back(); hs[hi].pop_back();
29            while (ex[u] > 0) { // discharge u
30                if (cur[u] == g[u].size()) {
31                    h[u] = 1e9;
32                    fore(i,0,SZ(g[u])){
33                        auto &e = g[u][i];
34                        if (e.f && h[u] > h[e.to]+1) h[u] = h[e.to]
                            +1, cur[u] = i;
35                    }
36                    if(hi==n)break;
37                    if(++co[h[u]] && !--co[hi] && hi<n){
38                        fore(i,0,n)if(hi < h[i] && h[i] < n){
39                            --co[h[i]]; h[i] = n + 1;
40                        }
41                    }
42                    hi = h[u];
43                } else if (g[u][cur[u]].f && h[u] == h[g[u][cur
                    [u]].to]+1) {
44                    add_flow(g[u][cur[u]], min(ex[u], g[u][cur[u]
                        ].f));
45                } else ++cur[u];
46            }
47            while (hi>=0 && hs[hi].empty()) --hi;
48        }
49        return -ex[s];
50    }
51    void push(edge &e, tf amt){
52        if(e.f < amt) amt=e.f;
53        e.f-=amt; ex[e.to]+=amt;
54        g[e.to][e.rev].f+=amt;
55        ex[g[e.to][e.rev].to]-=amt;
56    }
57    void relabel(int vertex){
58        tc newHeight = -INFCOST;
59        fore(i,0,SZ(g[vertex])){
60            edge const&e = g[vertex][i];
61            if(e.f && newHeight < h[e.to]-e.c){
62                newHeight=h[e.to]-e.c; cur[vertex]=i;
63            }
64        }
65        h[vertex] = newHeight - eps;
66    }
67    pair<tf, tc> minCostMaxFlow(){
68        tc retCost = 0;
69        fore(i,0,n)for(edge &e:g[i])retCost+=e.c*(e.f);
70        tf retFlow = max_flow();
71        h.assign(n, 0); ex.assign(n, 0); isq.assign(n, 0)
            ; cur.assign(n,0);
72        queue<int> q;
73        for(;eps;eps>=scale){
74            fill(cur.begin(), cur.end(), 0);
75            fore(i,0,n) for(auto &e:g[i])
76                if(h[i] + e.c - h[e.to] < 0 && e.f) push(e, e.f
                    );
77            fore(i,0,n) if(ex[i]>0)q.push(i),isq[i]=1;

```

```

78 while(!q.empty()){
79   int u=q.front();q.pop();
80   isq[u]=0;
81   while(ex[u]>0){
82     if(cur[u] == g[u].size()) relabel(u);
83     for(int &i=cur[u], max_i = g[u].size();i<
84           max_i;++i){
85       edge &e=g[u][i];
86       if(h[u] + e.c - h[e.to] < 0){
87         push(e, ex[u]);
88         if(ex[e.to]>0 && isq[e.to]==0) q.push(e.to)
89           , isq[e.to]=1;
90         if(ex[u]==0) break;
91       }
92     }
93   }
94   if(eps>1 && eps>>scale==0)eps = 1<<scale;
95 }
96 fore(i,0,n) for(edge &e:g[i])retCost-=e.c*(e.f);
97 return make_pair(retFlow, retCost/2/n);
98 }
99 tf getFlow(edge const &e){
100   return g[e.to][e.rev].f;
101 }

```

6.7 Useful stuff

- Rebuild cover $(A - Z) \cup (B \cap Z)$ Z = unmatched from L + reachable by alternating paths

```

1 vector<bool> L, R;
2 void dfs2(int x, int part) {
3   if(!part&&L[x]){L[x]=0;for(auto v:g[x])dfs2(v,!
4     part);}
5   else if(part&&!R[x])R[x]=1,dfs2(mt[x],!part);
6 }
7 void cover() {L.assign(n, 1); R.assign(m, 0);
8   fore(i,0,n)if(mt2[i]<0)dfs2(i,0);
9 }

```

- Vertices in all matchings. Orient matched edges $L \rightarrow R$, and others $R \rightarrow L$, v can be omitted if:

- v is unmatched
- v can reach an unmatched vertex ON ITS SIDE

- Circulation with demands

- add new source s and sink t
- if $d(v) < 0$, add edge $(s, v, -d(v))$
- if $d(v) > 0$, add edge $(v, t, d(v))$
- There is circulation if all s, t arcs saturate

- Circulation with demands and lowerbounds (l)

- new $cap(e) = cap(e) - l(e)$
- new $d(v) = d(v) + \text{sum of } l(e) \text{ out} - \text{sum of } l(e) \text{ in}$

- Flow with edge demands

- add new source s' and sink t'
- add new edge (t, s, ∞)
- for each $add_demand(x, y, d, c)$ (add edge with demand d and capacity c , $d \leq c$)

- * add edge (s', y, d)
- * add edge (x, t', d)
- * add edge $(x, y, c - d)$

- run Dinic from s' to t' and check if s' is saturated
- for max flow, run again Dinic from s to t

7 Other

7.1 Mo's algorithm

```

1 int n,sq,nq; // size, sqrt(size), #queries
2 struct qu{int l,r,id;}; // HILT -> ll ord;
3 qu qs[MAXN]; ll ans[MAXN]; // ans[i] = answer to
4   ith query
5 /* HILT:
6 const int HOLG = 18, HON = 1 << HOLG; // HON*HON is
7   the size of the square (HON >= MAXN)
8 ll hilbertOrder(int x, int y){
9   ll rx, ry, s, d = 0;
10  for(s=HON/2; s>0; s/=2){
11    rx = (x&s)>0, ry = (y&s)>0;
12    d += s*s*((3*rx)^ry);
13    if(!ry){
14      if(rx) x = HON-1-x, y = HON-1-y;
15      swap(x, y);
16    }
17  }
18  return d;
19 }
20 bool qcomp(const qu &a, const qu &b) { return a.ord
21   < b.ord; }
22 */
23 bool qcomp(const qu &a, const qu &b){
24   if(a.l/sq!=b.l/sq) return a.l<b.l;
25   return (a.l/sq)&1?a.r<b.r:a.r>b.r;
26 }
27 void mos(){
28   fore(i,0,nq)qs[i].id=i; // HILT -> qs[i].ord =
29     hilbertOrder(qs[i].l, qs[i].r);
30   sq=sqrt(n)+.5; sort(qs,qs+nq,qcomp); int l=0,r=0;
31   init();
32   fore(i,0,nq){
33     qu q=qs[i];
34     while(l>q.l)add(--l);
35     while(r<q.r)add(r++);
36     while(l<q.l)remove(l++);
37     while(r>q.r)remove(--r);
38     ans[q.id]=get_ans();
39   }
40 }

```

7.2 1D1D DP optimization

```

1 // dp[i] = min_{j=i+1}^{n} f(i,j) + dp[j]
2 // monge inequality must hold for f
3 ll dp[MAXN];
4 ll cost(ll l, ll r){} // f(l,r) + dp[r]
5 ll opt1d1d(){
6   deque<ii>dq; dq.pb({n-1,n});
7   for(ll j=n-1;j>=0;j--){ // info hasta (j
8     auto opt=dq.back().snd; assert(dq.back().fst==j
9       );
10    dp[j]=cost(j,opt); if(!j)break; dq.pop_back();

```

```

10 if(!SZ(dq) || dq.back().fst<j-1)dq.pb({j-1,opt});
11 ll s=0;
12 while(SZ(dq)){
13     auto [i,opt]=dq.front();
14     if(cost(i,j)<cost(i,opt))s=i,dq.pop_front();
15     else break;
16 }
17 if(!SZ(dq)){assert(s==j-1);dq.pb({s,j});
18     continue;}
19 ll l=s,r=dq.front().fst-1;
20 while(l<=r){
21     ll m=(l+r)/2;
22     if(cost(m,j)<cost(m,dq.front().snd))l=m+1;
23     else r=m-1;
24 }
25 if(r>=0)dq.push_front({r,j});
26 }
27 return dp[0];
}

```

7.3 Other stuff

```

1 // Iterate all NON EMPTY submasks of m
2 for(int s=m;s=s-(s-1)&m) // Decreasing order
3 for(int s=0;s=s-m&m;) // Increasing order
4 // Iterate all n-bit bitmasks with k ones,
5 // increasing
6 ll mk=(1ll<<k)-1,r,c;
7 while(mk<=(1ll<<n)-(1ll<<(n-k))){
8     // Code here
9     if(!k)break;
10    c=mk&-mk,r=mk+c,mk=r|(((r^mk)>>2)/c);
11 }
12 //interactive tester
13 mkfifo fifo; (./solution < fifo) | (./interactor >
14     fifo)
15 ifstream inpu; inpu.open("pre", ios::in); // file
16 //input
17 //generator
18 from os import system
19 system("./a<in>o")
20 x=open("o","r").read().strip().split()
21 //dp optimizations
22 d&c: opt(i,j) <= opt(i,j+1)
23 knuth: opt(i,j-1) <= opt(i,j) <= opt(i+1,j)
24 //Pick: A=I+B/2-1 (area, inside, border)
25 auto rng=mt19937(time(0));
26 uniform_(int|real)_distribution<int>(a,b)(rng);
27 #pragma GCC optimize("Ofast") // may lead to
28 //precision errors
29 #pragma GCC optimize("O3,unroll-loops")
30 #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
31 #include <tr2/dynamic_bitset> // dynamic bitset<> b
32 using namespace tr2; // bugged shifts in GCC <=
33 14.2 !!

```

```

29 x1 MOD=212345678987654321LL,P=1777771,PI
30 =106955741089659571LL; //hash consts
31 #include<ext/pb_ds/assoc_container.hpp> // extended
32 set
33 #include<ext/pb_ds/tree_policy.hpp>
34 using namespace __gnu_pbds;
35 typedef tree<int,null_type,less<int>,rb_tree_tag,
36     tree_order_statistics_node_update>ordered_set;
37 gp_hash_table<ll,ll> ht; // faster hash table, same
38 //includes as extended set
39 ull splitmix64(ull x) { // splitmix
40     x += R1; // R1, R2, R3, random constant 64-bit
41     //odd numbers
42     x = (x ^ (x >> 30)) * R2;
43     x = (x ^ (x >> 27)) * R3;
44     return x ^ (x >> 31);
45 }
46 from sympy.ntheory.modular import crt # python crt

```

7.4 Max number of divisors up to 10^n

```

1 (0,1,1) (1,4,6) (2,12,60) (3,32,840) (4,64,7560)
2 (5,128,83160) (6,240,720720) (7,448,8648640)
3 (8,768,73513440) (9,1344,735134400)
4 (10,2304,6983776800) (11,4032,97772875200)
5 (12,6720,963761198400) (13,10752,9316358251200)
6 (14,17280,97821761637600)
7 (15,26880,866421317361600)
8 (16,41472,8086598962041600)
9 (17,64512,74801040398884800)
10 (18,103680,897612484786617600)

```

7.5 Record Prime Gaps up to 10^n

```

1 (1, 4, 7) (2, 8, 89) (3, 20, 887) (4, 36, 9551)
2 (5, 72, 31397) (6, 114, 492113) (7, 154, 4652353)
3 (8, 220, 47326693) (9, 282, 436273009)
4 (10, 354, 4302407359) (11, 464, 42652618343)
5 (12, 540, 738832927927) (13, 674, 7177162611713)
6 (14, 804, 90874329411493)
7 (15, 906, 218209405436543)
8 (16, 1132, 1693182318746371)
9 (17, 1220, 80873624627234849)
10 (18, 1442, 804212830686677669)
11 (19, 1510, 6787988999657777797)

```

7.6 Ideas

Meet in the Middle; Gauss Elimination (Z_p); Interpolate;
Centroid; HLD; SQRT/Mos; Biconnected/AP/Bridge; Matrix
Exp; Offline; D&C; EulerTour; DP
Optimization(Aliens,Knuth,etc); SparseTable; Strings; 2SAT;
Brute Force; 2^k jumps; Small-to-large;randomize;ver
diferencias;berlekamp;
!!! INICIALIZAR, REVISAR COMENTARIOS, COTAS,
INT128/LL, VER N=1 !!!